

DESIGN OF SENSOR NETWORK WITH LONG DISTANCE COMMUNICATION FOR APPLICATION IN BUILDING ENERGY MANAGEMENT SYSTEM

*A Project Report submitted to
Jawaharlal Nehru Technological University
in partial fulfillment of the requirements for the award of Degree of*

**BACHELOR OF TECHNOLOGY
in
ELECTRONICS AND COMMUNICATION ENGINEERING**

Submitted by

L. SAI SREE

22831A0467

Under the Guidance of

Dr. S. Sivaiah

ASSISTANT PROFESSOR



**Department of ECE
Guru Nanak Institute of Technology
Ibrahimpattanam, Hyderabad, R.R. District – 501506**

APRIL, 2026

CERTIFICATE

This is to certify that the Major project Stage II entitled DESIGN OF SENSOR NETWORK WITH LONG DISTANCE COMMUNICATION FOR APPLICATION IN BUILDING ENERGY MANAGEMENT SYSTEMS is being submitted by Ms. SAISREE, bearing Roll No. 22831A0467, in partial fulfillment for the award of the Degree of Bachelor of the Technology in ELECTRONICS AND COMMUNICATION ENGINEERING to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by them under my guidance and supervision.

The results embodied in this Major project Stage II report have not been submitted to any other Institute for the award of any Degree or Diploma

Internal Guide

Dr. S. Sivaiah

Assistant Professor

Project Coordinator

Mrs. G. Praveena Reddy

Assistant Professor

Head of the Department

Dr. Anitha Swamidas

Professor

External Examiner

DECLARATION OF STUDENT

I, L. Sai Sree (22831A0467) hereby declare that the major project titled “Design of Sensor Network with Long Distance Communication for Application in Building Energy Management Systems” has been carried out by us as part of the requirements for the award of the Degree of Bachelor of Technology in the Department of Electronics and Communication Engineering at Guru Nanak Institute of Technology.

We confirm the following:

1. The project was undertaken by us under the supervision of our guide, Dr. S. Sivaiah, from the selection of the topic to the completion of the final report.
2. I have ensured that the results presented in the report are accurate and based on our original work.
3. To the best of our knowledge, the content of this report is free from plagiarism and adheres to ethical standards.
4. Each member of the team has contributed significantly and appropriately to the project work.
5. The project report has been prepared with diligence, ensuring clarity, accuracy, and adherence to academic standards.

I further declare that this report has not been submitted, to any other institution for the award of any degree or diploma.

L. Sai Sree
22831A0467

Signature Student

Date:24/04/2026

Place: Ibrahimpatnam , Hyderabad

DECLARATION OF GUIDE

I, Dr. S. Sivaiah, hereby declare that I have guided the major project titled “Design of Sensor Network with Long Distance Communication for Application in Building Energy Management Systems” undertaken by L. Sai Sree (22831A0467). This project was carried out towards the fulfillment of the requirements for the award of the Degree of Bachelor of Technology in the Department of Electronics and Communication Engineering at Guru Nanak Institute of Technology

As the guide, I confirm the following:

1. I have overseen the entire project process, from the selection of the project title to the submission of the final report.
2. I have reviewed and certified the accuracy and relevance of the results presented in the report.
3. The work carried out is original, free from plagiarism, and adheres to ethical guidelines.
4. The contributions of each student have been appropriately recognized and assessed.
5. The project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of Guide: Dr. S. Sivaiah

Signature of Guide

Date:24/04/2026

Place: Ibrahimpatnam, Hyderabad

Name of HOD: Dr. Anitha Swamidas

Signature of HOD

Department: ECE

Date:24/04/2026

Place: Ibrahimpatnam, Hyderabad

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to our internal guide, **Dr. S. Sivaiah, Assistant Professor**, Electronics and Communication Engineering, for his valuable guidance, encouragement, and continuous support throughout the duration of this project.

I am also thankful to **Dr. Anitha Swamidas, HOD**, Electronics and Communication Engineering, for her expert supervision and helpful suggestions, which contributed significantly to the successful completion of this project.

I would like to express our sincere gratitude to **Dr. S. P. Yadav, Dean Academics**, for his valuable support and guidance. I would like to express my profound sense of gratitude to **Dr. B. Venkata Ramana Reddy, Principal**, for his constant and valuable guidance.

I would like to express our deep sense of gratitude to **Dr. H. S. Saini, Managing Director**, Guru Nanak Group of Institutions for his tremendous support, encouragement, and inspiration.

I would also like to thank the faculty members of the **Electronics and Communication Engineering** and the Lab Technicians for their assistance and cooperation during the practical work of our project.

I am grateful to our friends and well-wishers for their encouragement, collaboration, and useful feedback throughout the project journey.

Lastly, I sincerely thank our parents for their constant support, patience, and motivation, which helped me complete this project successfully.

L. Sai Sree

22831A0467

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
CERTIFICATE	i
DECLARATION OF STUDENT	ii
DECLARATION OF GUIDE	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	vii
LIST OF FIGURES	viii
LIST OF TABLES	ix
ABBREVIATIONS/ NOTATIONS/ NOMENCLATURE	x
1. INTRODUCTION	1
1.1 GENERAL	1
1.2 EXISTING SYSTEM	3
1.3 PROPOSED SYSTEM	3
2. LITERATURE SURVEY	4
3. DESIGN AND DEVELOPMENT	7
3.1 INTRODUCTION TO EMBEDDED SYSTEM	7
3.2 SYSTEM ARCHITECTURE	10
3.3 BLOCK DIAGRAM	14
3.4 MODULES	15
3.4.1 POWER SUPPLY	15
3.4.2 TRANSFORMER	15
3.4.3 RECTIFIER	16
3.4.4 BRIDGE RECTIFIER	16
3.4.5 VOLTAGE REGULATORS	17
3.4.6 MICROCONTROLLER	18
3.4.7 RASPBERRY PI PICO PINOUT	21
3.4.8 OLED	27
3.4.9 VOLTAGE SENSOR	30
3.4.10 CURRENT SENSOR	32

3.4.11 IR SENSOR	33
3.4.12 ZIGBEE	35
3.4.13 ULN2003	38
3.4.14 RELAY	39
3.4.15 BUZZER	41
3.5 SOFTWARE SPECIFICATIONS	42
3.5.1 TOOLS	49
3.5.2 SKETCHBOOK	50
3.5.3 UPLOADING	50
3.5.4 LIBRARIES	51
3.5.5 SERIAL MONITOR	52
3.5.6 PREFERENCES	52
3.5.7 BOARDS	53
3.6 IMPLEMENTATIONS	53
3.7 PHOTOGRAPHS	57
4. RESULT AND DESCRIPTION	58
5. CONCLUSION	59
REFERENCE	60

ABSTRACT

This paper presents the design of a sensor network capable of transmitting data over long distances (up to 1000 meters) achieving reliable data, with the aim of improving energy consumption efficiency in Building Energy Management Systems (BEMS). The processing system uses microcontroller, the communication protocol proposed is Zigbee and the IoT (Internet of Things) platform to remotely view and control sensor data is Ubidots. The main contribution of this project is the design of environmental sensor node with high range of communications capabilities using Zigbee devices and found the right combination of the different communication factors (distance, spreading factor, bandwidth, and power). The project seeks to contribute to the field of building automation and long-distance communications allowing greater efficiency in energy consumption, reducing costs and minimizing environmental impact. Finally, communication validation is planned.

LIST OF FIGURES

FIGURE	TITLE	PAGE NUMBER
1.	Layered Structure of Embedded System	11
2.	Architecture of Embedded system	12
3.	Block Diagram	14
4.	Block Diagram of Receiver section	14
5.	Block diagram of Power Supply	15
6.	Bridge Rectifier	16
7.	Output waveform of DC	17
8	Regulator	17
9	Circuit Diagram of Power Supply	18
10	Raspberry Micro Controller	18
11	Raspberry Pi Pico Pinout	21
12	USB port on the end of Pico	24
13	Bootsel	25
14	Accessible Debugging Pins	25
15	OLEDs	28
16	IR Sensor	33
17	Logic diagram of ULN2003	39
18	Relay	40
19	Representation of Relay	41
20	Buzzer	42
21	Photograph of Result	57

LIST OF TABLES

TABLE	TITLE	PAGE NUMBER
1.	Pi Pico Power Pins	23
2.	Pico Ground Pins	24
3.	Pico Pin Description	26
4.	Name and Function of pin OLED	29
5.	Zigbee comparison	37

ABBREVIATIONS

WPAN	Wireless Personal Area Network
DSSS	Direct Sequence Spread Spectrum
AES	Advanced Encryption Standard
ISDN	Integrated Services Digital Network
GPIO	General Purpose Input Output
PWM	Pulse Width Modulation
LED	Light Emitting Diode
LCD	Liquid Crystal Display
OLED	Organic Light Emitting Diode
USB	Universal Serial Bus
IR	Infrared
UPS	Uninterruptible Power Supply
BEMS	Building Energy Management System
SPI	Serial Peripheral Interface
I2C	Inter-Integrated Circuit

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Smart cities make full use of information technology systems to increase their operational efficiency and improve the quality of public services (Zanella et al., 2014). In this context, smart building is an important element of the smart city concept. It is defined as a building with high energy efficiency, which uses intelligent management of energy consuming equipment and also energy producers (Snoonian, 2003). Furthermore, several government initiatives have focused on energy efficiency. One such initiative is NASA's renovation by replacement (RbR), which aims to replace old and inefficient buildings with new energy-efficient buildings. European Union (EU) also faces an energy challenge because it depends on energy imports and has moderate energy reserves. EU is considered as a leader when it comes to addressing climate change. Climate change also requires reducing greenhouse gas emissions, and therefore improving energy efficiency has paramount significance. To reduce energy consumption in buildings, it is necessary to develop smart solutions able to provide systems with very high energy efficiency. One of smart solutions could be deploying sensor networks for equipment monitoring. A sensor network consists of small, inexpensive, spatially distributed devices which cooperate to monitor physical environment (Akyildiz et al., 2002). It is composed of several nodes that can use different communication media to transmit and receive data through channels such as wired channel, wireless channel, optical channel, etc. Wireless communications are the most used in modern sensor networks because in a large-scale network, it becomes impossible to link nodes by wired channels (Yick et al., 2008). Sensor network lifecycle involves different steps including network sizing, software and hardware development, analyses and simulations, physical deployment, and data exploitation. This paper aims to answer issues related to pollution and energy efficiency of equipment in buildings. To achieve this objective, a sensor network is used to implement smart building system. We notice that several

smart building systems can be found in the literature (Hernandez et al., 2014). However, some systems that use sensor network for smart building do not cover all steps of sensor network lifecycle such as data exploitation (Mainwaring et al., 2002). Often, these systems are limited to a single communications technology and it does not cover heterogeneous networks. Furthermore, they are specific system therefore it is difficult to adapt them for other applications. Some researchers have already discussed smart building systems. However, most of them are in the proposal phase, and a few of them involve practical implementation (Morvaj et al., 2011; Wilson et al., 2017; Jacobsson et al., 2016). Some industrial smart building systems are based on platforms such as Kimo KTH-CO2 developed by Testoon (2015). However, this platform is relatively cost and very specific so it is difficult to adapt it for other scenarios. In addition, this platform must be connected to a computer with a wired link in order to retrieve the measurements which may not be easy or possible regarding the physical environment. Some smart building projects are based on assumptions from laboratory or test bed controlled environments, which may be totally different from a real environment (Gezer et al., 2011). Furthermore, they did not provide a methodology or road map to design smart building system. Compared to these systems, we are looking to cover all steps of sensor network lifecycle from network sizing to data exploitation. The developed network is based on a generic solution with several communication technologies including ZigBee and LoRa (heterogeneous network), a flexible architecture for reconfiguration purposes and also with variable acquisition frequencies, an optimised network based on sensors with low energy consumption, and a low cost system deployed in real environment. We can summarise the main contributions of this paper in two parts:

- We developed a sensor network for smart building to study energy efficiency and pollution in buildings. This development includes hardware design of a new sensor board, programming of sensor and gateway nodes, and development of data exploitation interfaces.
- As a feedback of our sensor network development, we highlighted several challenges related to energy constraints, large-scale network, complexity of

development, costs, etc. To overcome these challenges, we introduce a new methodology based on a model driven approach that takes into account the constraints of energy, reduces complexity related to sensor network design, and decreases development costs. A draft of this approach is implemented using a generic modelling environment. The remainder of the paper is organised as follows. Section 2 gives brief description of the proposed sensor network architecture. In Section 3, the design of our sensor node is presented. Sections 4 and 5 provide more details on gateway nodes and storage interfaces, respectively. Then, we highlight challenges related to our sensor network development in Section 6. Section 7 presents a new model driven approach to address these challenges, and finally conclusions and perspectives are provided in Section 8.

1.2 EXISTING SYSTEM

Current building energy management systems often use short-range wireless networks or wired connections, which can limit coverage in large buildings and increase installation complexity and cost. These systems may struggle to collect real-time data from distant or hard-to-reach areas, reducing their overall efficiency.

1.3 PROPOSED SYSTEM

The proposed system introduces a sensor network with long-distance wireless communication capabilities, enabling seamless data transmission across large buildings or multi-floor structures. This enhances real-time monitoring and control of energy usage, improves system scalability, and reduces installation and maintenance costs, leading to more efficient and smarter energy management.

CHAPTER 2

LITERATURE SURVEY

2.1 Title: Understanding the Limits of LoRaWAN

Author: Adelantado, F. et al.

Year: 2022

Description:

This seminal paper provides a rigorous, technical analysis of the LoRaWAN protocol, a leading LPWAN (Low-Power Wide-Area Network) technology. It investigates fundamental limits in capacity, coverage range, and scalability, modeling how these constraints impact large-scale network deployments.

Advantages:

The work offers critical, data-driven insights for network planning, confirming LoRaWAN's exceptional link budget for long-range (several km in urban areas) and ultra-low power operation, making it theoretically ideal for low-data-rate BEMS applications like periodic meter reading.

Disadvantages:

It clearly outlines key limitations, including very low data rates, strict duty cycle regulations limiting channel access, and scalability challenges in dense node deployments, which can constrain applications requiring frequent data updates or real-time control.

2.2 Title: A Comprehensive Survey on LPWAN Technologies for Smart Cities

Author: Raza, U., Kulkarni, P., & Sooriyabandara, M.

Year: 2023

Description:

This survey offers a comparative analysis of major LPWAN technologies, including LoRaWAN, Sigfox, and NB-IoT, within the smart city context. It evaluates them based on key parameters like range, data rate, power consumption, and network topology.

Advantages:

It provides an excellent, side-by-side comparison for BEMS designers, highlighting the trade-offs between proprietary (LoRa/Sigfox) and cellular (NB-IoT) LPWANs. The paper effectively positions these technologies as solutions for cost-effective, city-scale sensor coverage.

Disadvantages:

As a high-level survey, it lacks implementation-specific details or real-world BEMS case study performance data. The rapid evolution of these technologies also means some specifics may now be dated.

2.3. Title: NB-IoT System for M2M Communication

Author: Nokia Bell Labs (Ratasuk, R. et al.)

Year: 2023

Description:

This paper details the design and capabilities of Narrowband IoT (NB-IoT), a 3GPP-standardized LPWAN technology. It explains its deployment modes (in-band, guard-band, standalone), its optimization for massive Machine-Type Communication (mMTC), and its link performance.

Advantages:

Highlights NB-IoT's strengths: superior deep indoor penetration due to high transmit power and licensed spectrum, high network reliability and security inherited from cellular infrastructure, and native support for massive numbers of devices, ideal for large property portfolios.

Disadvantages:

Notes the typically higher module cost compared to non-cellular LPWANs, potential latency issues, and dependency on mobile network operators, which may lead to ongoing subscription fees and less deployment control for the building owner.

2.4. Title: Design and Implementation of a Hybrid Wireless Sensor Network for Large-Scale Building Monitoring**Author:** Kim, Y.-S., et al.**Year:**2024**Description:**

This work presents a practical, multi-tier architecture combining short-range wireless (Zigbee/Bluetooth Low Energy) for intra-building sensor clustering with a long-distance LPWAN (LoRaWAN) gateway for backhaul to a cloud server.

Advantages:

The hybrid design leverages the best of both worlds: high-density, low-cost sensor nodes using short-range protocols and a single, long-distance link per building/cluster for efficient data aggregation and transmission. It reduces the overall system cost and gateway complexity.

Disadvantages:

Introduces design complexity in managing two distinct network stacks and requires a gateway device, which is a single point of failure for its cluster. Network latency can be higher due to the multi-hop intra-building routing prior to backhaul.

CHAPTER 3

DESIGN AND DEVELOPMENT

3.1 INTRODUCTION TO EMBEDDED SYSTEM

An embedded system can be defined as a computing device that does a specific focused job. Appliances such as the air-conditioner, VCD player, DVD player, printer, fax machine, mobile phone etc. are examples of embedded systems. Each of these appliances will have a processor and special hardware to meet the specific requirement of the application along with the embedded software that is executed by the processor for meeting that specific requirement. The embedded software is also called “firm ware”. The desktop/laptop computer is a general-purpose computer. You can use it for a variety of applications such as playing games, word processing, accounting, software development and so on.

In contrast, the software in the embedded systems is always fixed listed below:

- Embedded systems do a very specific task, they cannot be programmed to do different things. . Embedded systems have very limited resources, particularly the memory. Generally, they do not have secondary storage devices such as the CDROM or the floppy disk. Embedded systems have to work against some deadlines. A specific job has to be completed within a specific time. In some embedded systems, called real-time systems, the deadlines are stringent. Missing a deadline may cause a catastrophe-loss of life or damage to property. Embedded systems are constrained for power. As many embedded systems operate through a battery, the power consumption has to be very low.
- Some embedded systems have to operate in extreme environmental conditions such as very high temperatures and humidity.

Application Areas

Nearly 99 per cent of the processors manufactured end up in embedded systems. The embedded system market is one of the highest growth areas as these systems are used in very market segment- consumer electronics, office automation, industrial automation, biomedical engineering, wireless

communication, data communication, telecommunications, transportation, military and so on.

Consumer appliances

At home we use a number of embedded systems which include digital camera, digital diary, DVD player, electronic toys, microwave oven, remote controls for TV and air-conditioner, VCO player, video game consoles, video recorders etc. Today's high-tech car has about 20 embedded systems for transmission control, engine spark control, air-conditioning, navigation etc. Even wristwatches are now becoming embedded systems. The palmtops are powerful embedded systems using which we can carry out many general-purpose tasks such as playing games and word processing.

Office Automation

The office automation products using embedded systems are copying machine, fax machine, key telephone, modem, printer, scanner etc.

Industrial Automation

Today a lot of industries use embedded systems for process control. These include pharmaceutical, cement, sugar, oil exploration, nuclear energy, electricity generation and transmission. The embedded systems for industrial use are designed to carry out specific tasks such as monitoring the temperature, pressure, humidity, voltage, current etc., and then take appropriate action based on the monitored levels to control other devices or to send information to a centralized monitoring station. In hazardous industrial environment, where human presence has to be avoided, robots are used, which are programmed to do specific jobs. The robots are now becoming very powerful and carry out many interesting and complicated tasks such as hardware assembly.

Medical Electronics

Almost every medical equipment in the hospital is an embedded system. These equipment's include diagnostic aids such as ECG, EEG, blood pressure measuring devices, X-ray scanners; equipment used in blood analysis, radiation,

colonoscopy, endoscopy etc. Developments in medical electronics have paved way for more accurate diagnosis of diseases.

Computer Networking

Computer networking products such as bridges, routers, Integrated Services Digital Networks

(ISDN), Asynchronous Transfer Mode (ATM), X.25 and frame relay switches are embedded systems which implement the necessary data communication protocols. For example, a router interconnects two networks. The two networks may be running different protocol stacks. The router's function is to obtain the data packets from incoming pores, analyze the packets and send them towards the destination after doing necessary protocol conversion. Most networking equipments, other than the end systems (desktop computers) we use to access the networks, are embedded systems.

Telecommunications

In the field of telecommunications, the embedded systems can be categorized as subscriber terminals and network equipment. The subscriber terminals such as key telephones, ISDN phones, terminal adapters, web cameras are embedded systems. The network equipment includes multiplexers, multiple access systems, Packet Assemblers Disassemblers (PADs), satellite modems etc. IP phone, IP gateway, IP gatekeeper etc. are the latest embedded systems that provide very low-cost voice communication over the Internet.

Wireless Technologies

Advances in mobile communications are paving way for many interesting applications using embedded systems. The mobile phone is one of the marvels of the last decade of the 20'h century. It is a very powerful embedded system that provides voice communication while we are on the move. The Personal Digital Assistants and the palmtops can now be used to access multimedia service over the Internet. Mobile communication infrastructure such as base station controllers, mobile switching centres are also powerful embedded systems.

Security

Security of persons and information has always been a major issue. We need to protect our homes and offices; and also the information we transmit and store. Developing embedded systems for security applications is one of the most lucrative businesses nowadays. Security devices at homes, offices, airports etc. for authentication and verification are embedded systems. Encryption devices are nearly 99 per cent of the processors that are manufactured end up in~ embedded systems. Embedded systems find applications in every industrial segment- consumer electronics, transportation, avionics, biomedical engineering, manufacturing, process control and industrial automation, data communication, telecommunication, defense, security etc. Used to encrypt the data/voice being transmitted on communication links such as telephone lines

Finance

Financial dealing through cash and cheques are now slowly paving way for transactions using smart cards and ATM (Automatic Teller Machine, also expanded as Any Time Money) machines. Smart card, of the size of a credit card, has a small micro-controller and memory; and it interacts with the smart card reader! ATM machine and acts as an electronic wallet. Smart card technology has the capability of ushering in a cashless society. Well, the list goes on. It is no exaggeration to say that eyes wherever you go, you can see, or at least feel, the work of an embedded system.

3.2 SYSTEM ARCHITECTURE

Every embedded system consists of custom-built hardware built around a Central Processing Unit (CPU). This hardware also contains memory chips onto which the software is loaded. The software residing on the memory chip is also called the ‘firmware’. The embedded system architecture can be represented as a layered architecture as shown in Fig. The operating system runs above the hardware, and the application software runs above the operating system. The

same architecture is applicable to any computer including a desktop computer.

However, there are significant differences. It is not compulsory to have an operating system in every embedded system. For small appliances such as remote control units, air conditioners, toys etc., there is no need *for* an operating system and you can write only the software specific to that application. For applications involving complex processing, it is advisable to have an operating system. In such a case, you need to integrate the application software with the operating system and then transfer the entire software on to the memory chip. Once the software is transferred to the memory chip, the software will continue to run *for* a long time you don't need to reload new software.

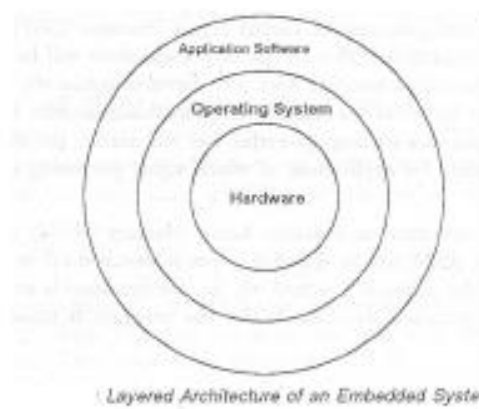


Fig 1: Layered structure of embedded system

Now, let us see the details of the various building blocks of the hardware of an embedded system. As shown in Fig. the building blocks are;

- Central Processing Unit (CPU)
- Memory (Read-only Memory and Random Access Memory)
- Input Devices
- Output devices
- Communication interfaces
- Application-specific circuitry

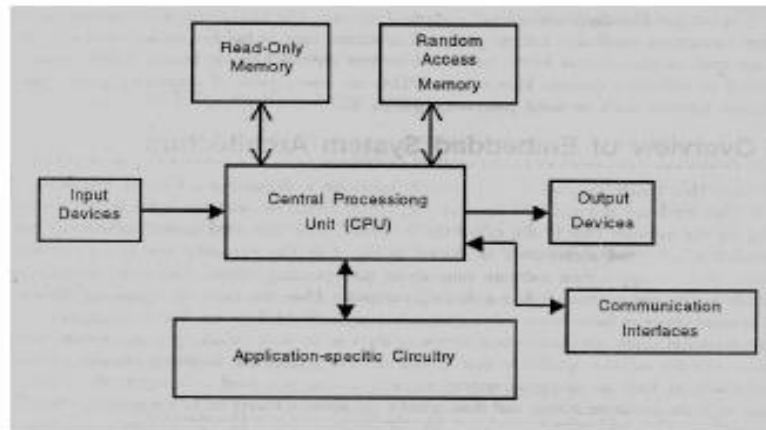


Fig 2: Architecture of embedded system

Central Processing Unit (CPU)

The Central Processing Unit (processor, in short) can be any of the following: microcontroller, microprocessor or Digital Signal Processor (DSP). A microcontroller is a low-cost processor. Its main attraction is that on the chip itself, there will be many other components such as memory, serial communication interface, analog-to digital converter etc. So, for small applications, a microcontroller is the best choice as the number of external components required will be very less. On the other hand, microprocessors are more powerful, but you need to use many external components with them. DSP is used mainly for applications in which signal processing is involved such as audio and video processing.

Memory

The memory is categorized as Random Access Memory (RAM) and Read Only Memory (ROM). The contents of the RAM will be erased if power is switched off to the chip, whereas ROM retains the contents even if the power is switched off. So, the firmware is stored in the ROM. When power is switched on, the processor reads the ROM; the program is executed.

Input Devices

Unlike the desktops, the input devices to an embedded system have very limited capability. There will be no keyboard or a mouse, and hence interacting with the embedded system is no easy task. Many embedded systems will have a small keypad-you press one key to give a specific command. A keypad may be used to input only the digits. Many embedded systems used in process control do not have any input device *for* user interaction; they take inputs *from* sensors or transducers and produce electrical signals that are in turn fed to other systems.

Output Devices

The output devices of the embedded systems also have very limited capability. Some embedded systems will have a *few* Light Emitting Diodes (LEDs) *to* indicate the health status of the system modules, or *for* visual indication of alarms. A small Liquid Crystal Display (LCD) may also be used to display *some* important parameters.

Communication Interfaces

The embedded systems may need to, interact with other embedded systems at they may have to transmit data to a desktop. To facilitate this, the embedded systems are provided with one or a *few* communication interfaces such as RS232, RS422, RS485, Universal Serial Bus (USB), IEEE 1394, Ethernet etc.

3.3 BLOCK DIAGRAM

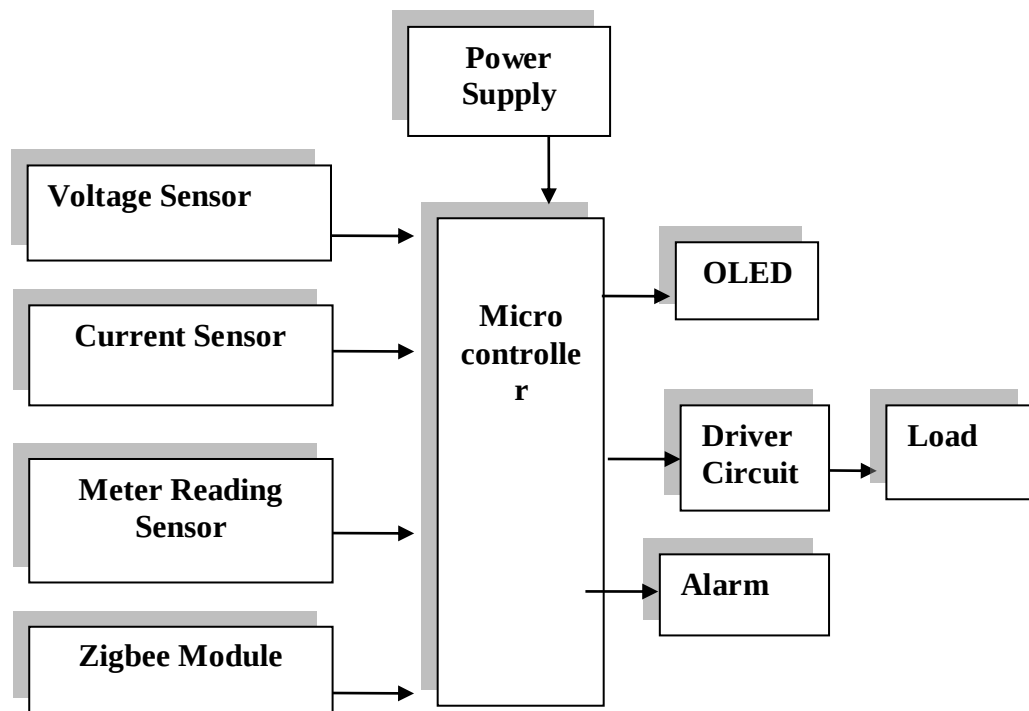


Fig 3: Block diagram

Receiver Section

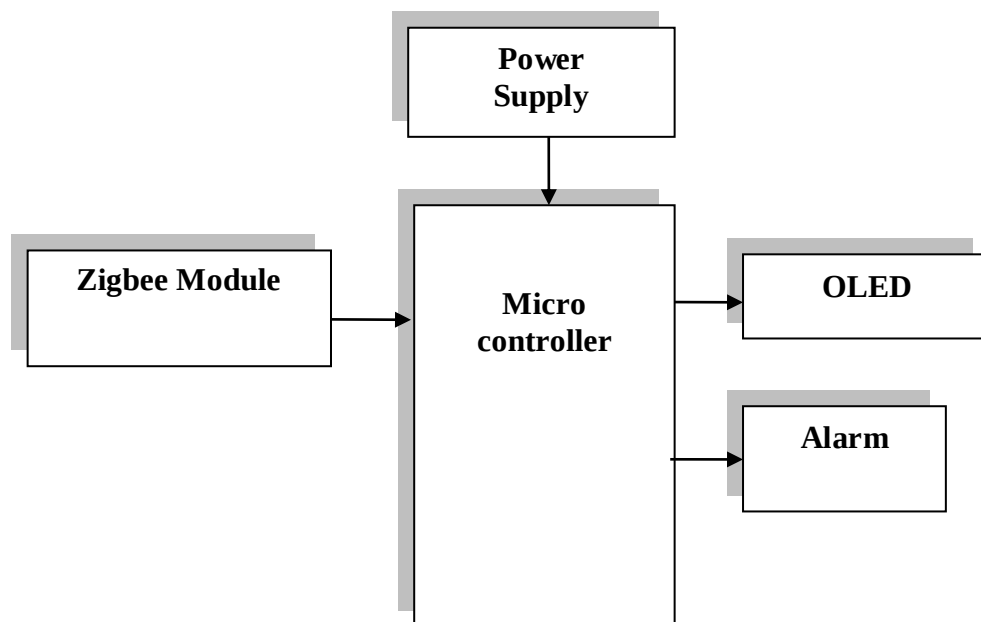


Fig 4: Block diagram of Receiver section

3.4 MODULES

3.4.1 POWER SUPPLY

The power supply section is the section which provide +5V for the components to work. IC LM7805 is used for providing a constant power of +5V. The ac voltage, typically 220V, is connected to a transformer, which steps down that ac voltage down to the level of the desired dc output. A diode rectifier then provides a full-wave rectified voltage that is initially filtered by a simple capacitor filter to produce a dc voltage. This resulting dc voltage usually has some ripple or ac voltage variation. A regulator circuit removes the ripples and also retains the same dc value even if the input dc voltage varies, or the load connected to the output dc voltage changes. This voltage regulation is usually obtained using one of the popular voltage regulator IC units.

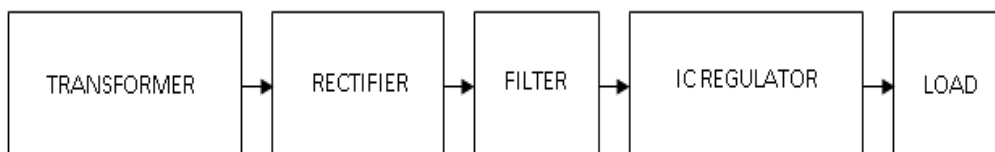


Fig 5: Block Diagram of Power Supply

3.4.2 TRANSFORMER

Transformers convert AC electricity from one voltage to another with little loss of power. Transformers work only with AC and this is one of the reasons why mains electricity is AC.

Step-up transformers increase voltage, step-down transformers reduce voltage. Most power supplies use a step-down transformer to reduce the dangerously high mains voltage (230V in India) to a safer low voltage. The input coil is called the primary and the output coil is called the secondary. There is no electrical connection between the two coils; instead they are linked by an alternating magnetic field created in the soft-iron core of the transformer. Transformers waste very little power so the power out is (almost) equal to the power in. Note that as voltage is stepped down current is stepped up. The transformer will step down the power supply voltage (0-230V) to (0- 6V) level. Then the secondary

of the potential transformer will be connected to the bridge rectifier, which is constructed with the help of PN junction diodes. The advantages of using bridge rectifier are it will give peak voltage output as DC.

3.4.3 RECTIFIER

There are several ways of connecting diodes to make a rectifier to convert AC to DC. The bridge rectifier is the most important and it produces full-wave varying DC. A full-wave rectifier can also be made from just two diodes if a centre-tap transformer is used, but this method is rarely used now that diodes are cheaper. A single diode can be used as a rectifier but it only uses the positive (+) parts of the AC wave to produce half-wave varying Dc

3.4.4 BRIDGE RECTIFIER

When four diodes are connected as shown in figure, the circuit is called as bridge rectifier. The input to the circuit is applied to the diagonally opposite corners of the network, and the output is taken from the remaining two corners. Let us assume that the transformer is working properly and there is a positive potential, at point A and a negative potential at point B. the positive potential at point A will forward bias D3 and reverse bias D4.

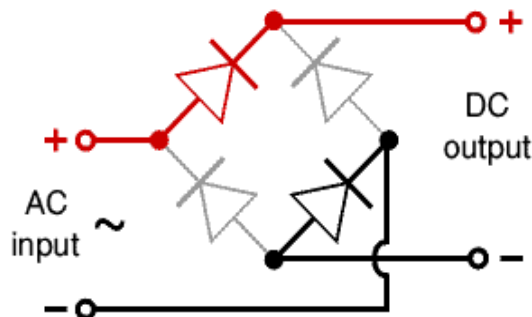


Fig 6: Bridge Rectifier

The negative potential at point B will forward bias D1 and reverse D2. At this time D3 and D1 are forward biased and will allow current flow to pass through them; D4 and D2 are reverse biased and will block current flow.

One advantage of a bridge rectifier over a conventional full-wave rectifier is that with a given transformer the bridge rectifier produces a voltage output that

is nearly twice that of the conventional full-wave circuit.

- i. The main advantage of this bridge circuit is that it does not require a special centre tapped transformer, thereby reducing its size and cost.
- ii. The single secondary winding is connected to one side of the diode bridge network and the load to the other side as shown below.
- iii. The result is still a pulsating direct current but with double the frequency.

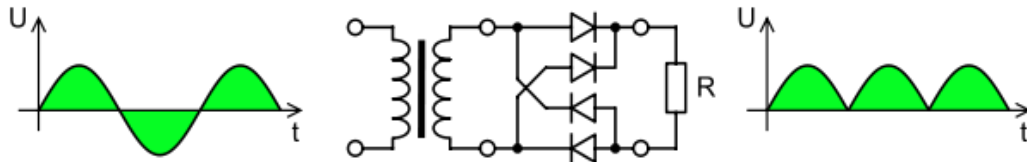


Fig 7: Output Waveform Of DC

3.4.5 VOLTAGE REGULATORS

Voltage regulators comprise a class of widely used ICs. Regulator IC units contain the circuitry for reference source, comparator amplifier, control device, and overload protection all in a single IC. IC units provide regulation of either a fixed positive voltage, a fixed negative voltage, or an adjustably set voltage. The regulators can be selected for operation with load currents from hundreds of milli amperes to tens of amperes, corresponding to power ratings from milli watts to tens of watts.

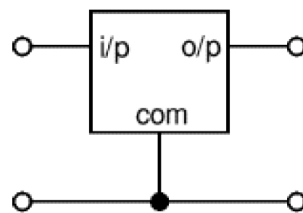


Fig 8: Regulator

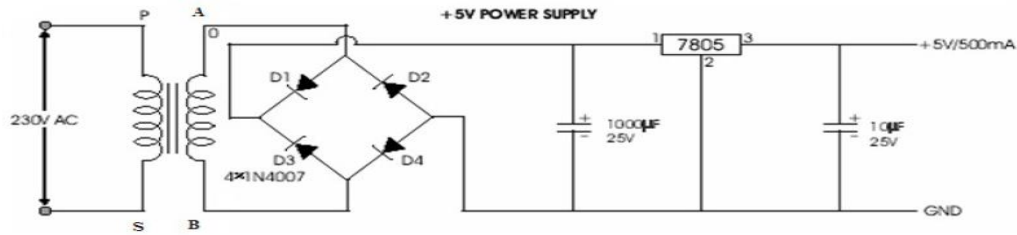


Fig 9: Circuit Diagram of Power Supply

3.4.6 MICROCONTROLLER



Fig 10: Raspberry micro controller

The Raspberry Pi Pico currently exists in two main versions:

Raspberry Pi Pico: The original one.

Raspberry Pi Pico W: A similar version, with an added wireless module that allows you to connect it to a Wi-Fi network on boot. The Raspberry Pi foundation changed single-board computing when they released the Raspberry Pi computer, now they're ready to do the same for microcontrollers with the release of the brand-new Raspberry Pi Pico. This low-cost microcontroller board features a powerful new chip, the RP2040, and all the fixin's to get started with embedded electronics projects at a stress-free price.

The Raspberry Pi Pico is a microcontroller board based on the Raspberry Pi RP2040 microcontroller chip. Like Raspberry Pi computers, Raspberry Pi Pico with 40 connections, along with a new debug connection enabling you to analyse your programs directly from another computer (typically by connecting it directly to the GPIO pins on a Raspberry Pi).

Raspberry Pi Pico is a brand new, low-cost, yet highly flexible development board designed around a custom-built RP2040 microcontroller chip designed by Raspberry Pi.

Raspberry Pi Pico – ‘Pico’ for short – features a dual-core Cortex-M0+ processor (the most energy-efficient Arm processor available), 264kb of SRAM, 2MB of flash storage, USB 1.1 with device and host support, and a wide range of flexible I/O options.

The Pico is 0.825" x 2" and can have headers soldered in for use in a breadboard or perfboard, or can be soldered directly onto a PCB with the castellated pads. There's 20 pads on each side, with groups of general purpose input-and-output (GPIO) pins interleaved with plenty of ground pins. All of the GPIO pins are 3.3V logic, and are not 5V-safe so stick to 3V! You get a total of **25 GPIO** pins (technically there are 26 but IO #15 has a special purpose and should not be used by projects), **3 of those can be analog inputs** (the chip has 4 ADC but one is not broken out). There are no true analog output (DAC) pins.

Features of Raspberry pi pico

- 21 mm × 51 mm form factor
- RP2040 microcontroller chip designed by Raspberry Pi in the UK
- Dual-core Arm Cortex-M0+ processor, flexible clock running up to 133 MHz

- 264KB on-chip SRAM
- 2MB on-board QSPI Flash
- 26 multifunction GPIO pins, including 3 analog inputs
- $2 \times$ UART, $2 \times$ SPI controllers, $2 \times$ I2C controllers, $16 \times$ PWM channels
- $1 \times$ USB 1.1 controller and PHY, with host and device support
- Supported input power 1.8–5.5V DC
- Operating temperature -20°C to $+85^{\circ}\text{C}$
- Castellated module allows soldering direct to carrier boards
- Drag-and-drop programming using mass storage over USB
- Low-power sleep and dormant modes
- Accurate on-chip clock
- Temperature sensor
- Accelerated integer and floating-point libraries on-chip

3.4.7 RASPBERRY PI PICO PINOUT

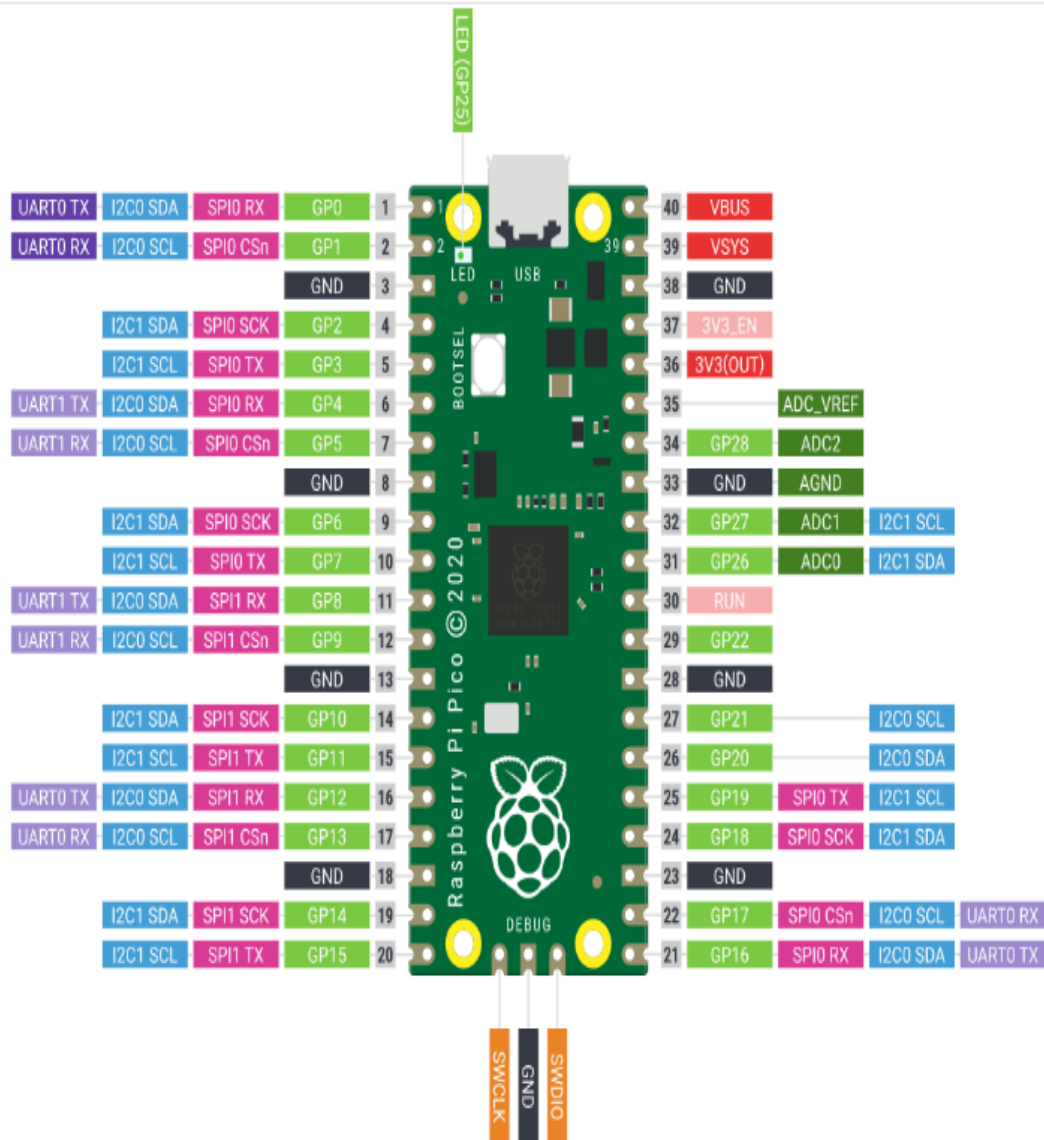


Fig 11: RASPBERRY PI PICO PINOUT

The Raspberry Pi only utilises 26 out of 30 GPIOs the RP2040 has to offer, 26 pins are exposed and an additional 27th pin can only be used for the onboard LED. The Raspberry Pi Pico's GPIO is powered from the onboard 3.3V rail and is therefore fixed at 3.3V. GPIO0 to GPIO22 are digital-only and GPIO 26-28 are able to be used either as digital GPIO or as an ADC input which can be done through software.

General Input and Output (GPIO) Pins on Pico

The Raspberry Pi only utilises 26 out of 30 GPIOs the RP2040 has to offer, 26 pins are exposed and an additional 27th pin can only be used for the

onboard LED. The Raspberry Pi Pico's GPIO is powered from the onboard 3.3V rail and is therefore fixed at 3.3V. GPIO0 to GPIO22 are digital-only and GPIO 26-28 are able to be used either as digital GPIO or as an ADC input which can be done through software.

The communication options this board has to offer are:

- $2 \times \text{SPI}$
- $2 \times \text{I2C}$
- $2 \times \text{UART}$
- $3 \times 12\text{-bit ADC}$
- $16 \times \text{controllable PWM channels}$

For peripherals, there are two I2C controllers, two SPI controllers, and two UARTs that are multiplexed across the GPIO - check the pinout for what pins can be set to which. There are 16 PWM channels, each pin has a channel it can be set to (ditto on the pinout).

While the RP2040 has lots of onboard RAM (264KB), it does not have built in FLASH memory. Instead that is provided by the external QSPI flash chip. On this board there is 2MB, which is shared between the program it's running and any file storage used by MicroPython or CircuitPython. When using C/C++ you get the whole flash memory, if using Python you will have about 1 MB remaining for code, files, images, fonts, etc

Power Pins on Pi Pico

Power pins are used for either powering the pico from the power source or powering the sensors and peripherals from the pico.

The Pi Pico can either be powered through a USB cable(via VBUS) or through a battery or other sources(via VSYS), when both the inputs are connected together, the input with higher potential supplies the power. For connecting 2 sources you need to connect it to VSYS in series with a Schottky diode.

Table 1: PI PICO Power Pins

VBUS	40	It is the 5V input from the micro-USB port, if there is no micro-USB attached to the pico, there won't be any power to the VBUS.
VSYS	39	It is the main system bus that supplies the 3.3V output to the RP2040 microprocessor and other IO pins. VSYS feeds power to the RT6150 SMPS which generates a fixed 3.3V output.
3V3_EN	37	it is the onboard enabled pin, it enables the 3.3V power supply on the pico board, it can be externally be used to turn the pico on or off.
3V3(Out)	36	It is the main pin supply pin to the RP2040, and its IO is generated by the SMPS, hence this pin can be used to enable output circuitry. Max permissible current is 300mA

Pico Ground Pins

There are 2 kinds of ground pins on the board, a digital ground and a ground pin for analogue IO. The differences are given below

Table 2: Pico ground pins

Name	Pin no.	Description
GND	3,8,13,18,23,28,33,38	All the ground pins in the boards are interconnected. All the devices, peripherals and sensors are connected to the ground pins to close the electrical circuit and to achieve a common reference

		ground throughout the circuit. The board has a total of 9 ground pins. 8 in GPIO's and 1 for Debugging
AGND	33	Also known as analog ground. It is intended for reference voltage for the ADCs and DACs. It is for the ground reference for GPIO26-29, a separate analog ground plane is running under these signals and terminating at this pin. In the absence of an ADC or DAC, this pin acts as a digital ground.

USB

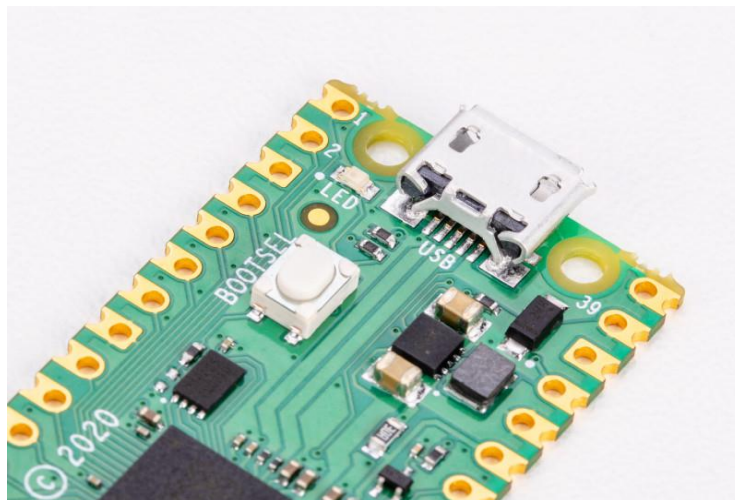


Fig 12: USB port on the end of pico

Bootsel

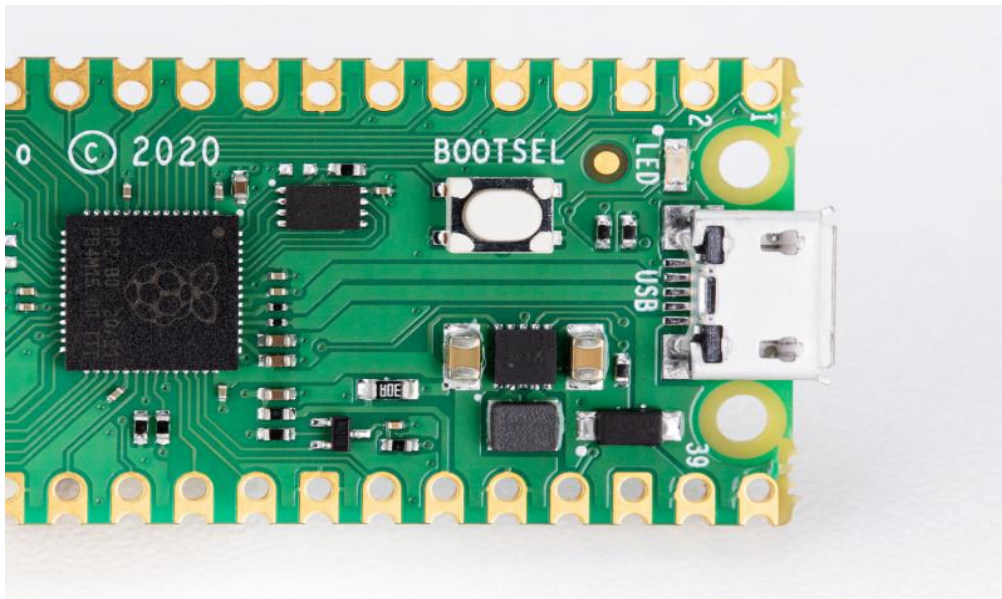


Fig 13: Bootsels

Hold the BOOTSEL (boot select) button when powering up Pico to put it into USB Mass Storage Mode. From here, you can drag-and-drop programs, created with C or MicroPython, into the RPI-RP2 mounted drive. Pico runs the program as soon as it is switched on (without BOOTSEL held down)

Labelling

Silkscreen labelling on the top provides orientation for the 40 pins, while a full pinout is printed on the rear

Debugging

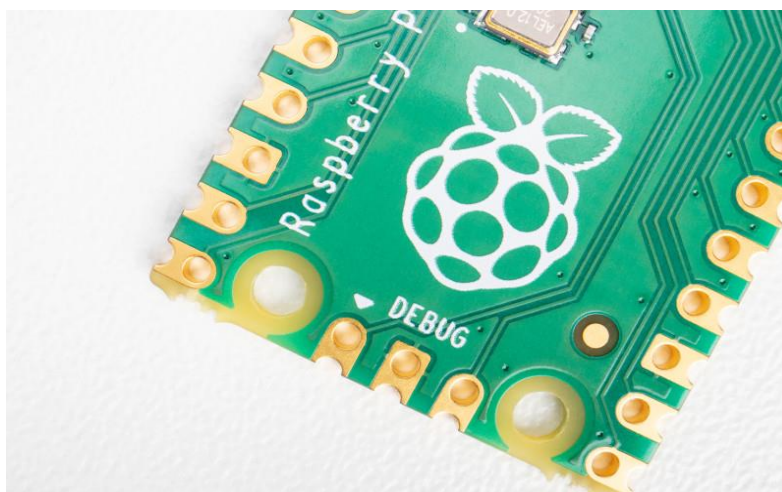


Fig 14: Accessible debugging pins

A Serial Wire Debug (SWD) header provides hardware debugging capabilities, letting you quickly track down problems in your programs

Table 3: Pico's Pins Description

Name	Description	Function
GP0-GP28	General-purpose input/output pins	Act as either input or output and have no fixed purpose of their own
GND	0 volts ground	Several GND pins around Pico to make wiring easier.
RUN	Enables or disables your Pico	Start and stop your Pico from another microcontroller.
GPxx_ADCx	General-purpose input/output or analog input	Used as an analog input as well as a digital input or output – but not both at the same time.
ADC_VREF	Analog-to-digital converter (ADC) voltage reference	A special input pin which sets a reference voltage for any analog inputs.
AGND	Analog-to-digital converter (ADC) 0 volts ground	A special ground connection for use with the ADC_VREF pin.
3V3(O)	3.3 volts power	A source of 3.3V power, the same voltage your Pico runs at internally, generated from the VSYS input.

Name	Description	Function
3v3(E)	Enables or disables the power	Switch on or off the 3V3(O) power, can also switches your Pico off.
VSYS	2-5 volts power	A pin directly connected to your Pico's internal power supply, which cannot be switched off without also switching Pico off.
VBUS	5 volts power	A source of 5

3.4.8 OLED (Organic Light Emitting Diodes)

OLED (Organic Light Emitting Diodes) is a flat light emitting technology, made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. OLEDs are emissive displays that do not require a backlight and so are thinner and more efficient than LCD displays (which do require a white backlight).

OLED displays are not just thin and efficient - they provide the best image quality ever and they can also be made transparent, flexible, foldable and even rollable and stretchable in the future. OLEDs represent the future of display technology!

OLED vs LCD

An OLED display have the following advantages over an LCD display:

- Improved image quality - better contrast, higher brightness, fuller viewing angle, a wider color range and much faster refresh rates.
- Lower power consumption.
- Simpler design that enables ultra-thin, flexible, foldable and transparent displays

- Better durability - OLEDs are very durable and can operate in a broader temperature range

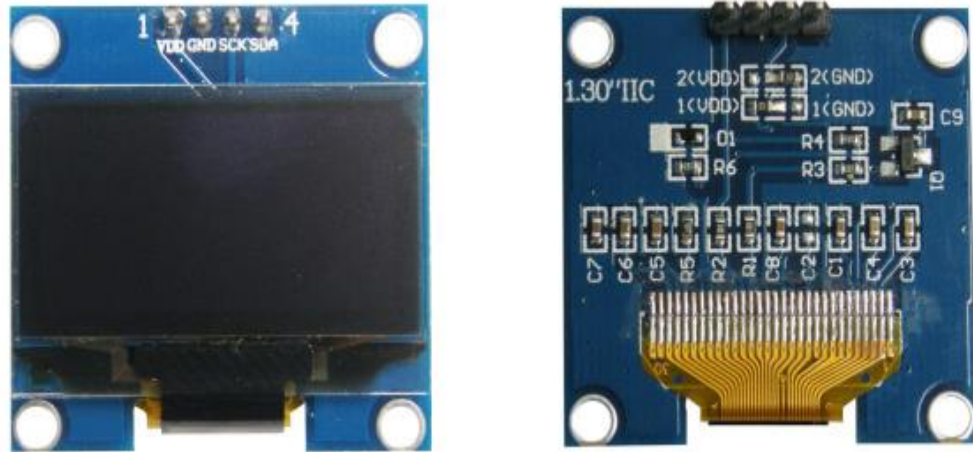


Fig 15: OLEDS

The future - flexible and transparent OLED displays

As we said, OLEDs can be used to create flexible and transparent displays. This is pretty exciting as it opens up a whole world of possibilities:

- Curved OLED displays, placed on non-flat surfaces
- Wearable OLEDs
- Foldable OLEDs and rollable OLEDs which can be used to create new mobile devices
- Transparent OLEDs embedded in windows or car windshields
- And many more we cannot even imagine today...

Flexible OLEDs are already on the market for many years (in smartphones, wearables and other devices) and since 2019, with the introduction of the Samsung Galaxy Fold, foldable devices are increasing in popularity. In 2019 LG also announced the world's first rollable OLED - its 65" OLED R TV that can roll into its base!

An OLED is made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. [Click here](#) for a more detailed view of the OLED technology.

OLEDs are organic because they are made from carbon and hydrogen. There's no connection to organic food or farming - although OLEDs are very efficient

and do not contain any bad metals - so it's a real green technology.

OLED is the best display technology - and indeed OLED panels are used today to create the most stunning TVs ever - with the best image quality combined with the thinnest sets ever. And this is only the beginning, as in the future OLED will enable large rollable and transparent TVs!

Currently the only company that produces OLED TV panels is LG Display. The Korean display maker is producing a wide range of OLED TV panels, offering these to LG Electronics, Panasonic, Sony, Philips and others.

OLED white lighting

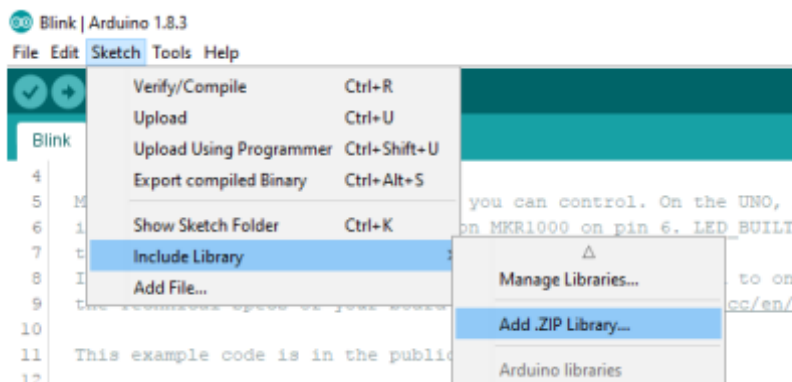
OLEDs can be used to create excellent light source. OLEDs offer diffuse area lighting and can be flexible, efficient, light, thin, transparent, color-tunable and more. OLEDs enable new designs and these devices emit healthier light compared to CFLs and LED lighting devices.

Specifications

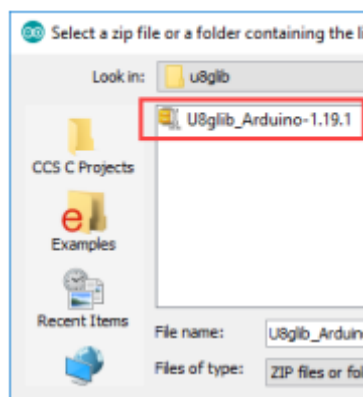
- ✓ Use CHIP No.SH1106
- ✓ Use 3.3V-5V POWER SUPPLY
- ✓ Graphic LCD 1.3" in width with 128x64 Dot Resolution
- ✓ White Display is used for the model OLED 1.3 I2C WHITE and blue Display is used for the model OLED 1.3 I2C BLUE
- ✓ Use I2C Interface
- ✓ Directly connect signal to Microcontroller 3.3V and 5V without connecting through Voltage Regulator Circuit
- ✓ Total Current when running together is 8 mA - PCB Size: 33.7 mm x 35.5 mm

Table 4: name and function of Pin OLED

Pin No.	Pin Name	Description
1	VDD	Pin Power Supply for LCD, using 3.3V-5V
2	GND	Pin Ground
3	SCK	Pin SCL of I2C Interface
4	SDA	Pin SDA of I2C Interface

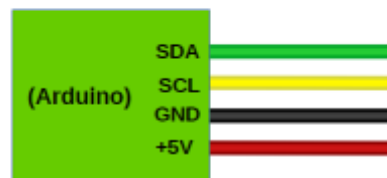


- Go to Folder Lib Arduino\u005Clib in CD-ROM; next



- Wire Circuit as shown in the picture below

ET-BASE AVR EASY328



3.4.9 VOLTAGE SENSOR

The Voltage Sensor is a device that converts voltage measured between two points of an electrical circuit into a physical signal proportional to the voltage.

Voltage sensor circuit is a combination of various electronic component by using which the accurate voltage value can be achieved. The major component utilize in voltage sensor are potentiometer & ADC.

Potentiometer

A potentiometer informally a pot, is a three-terminal resistor with a sliding contact that forms an adjustable voltage divider. If only two terminals are used, one end and the wiper, it acts as a variable resistor or rheostat.

A potentiometer measuring instrument is essentially a voltage divider used for measuring electric potential (voltage); the component is an implementation of the same principle, hence its name.

Analog To Digital Converter (ADC)

In many applications data collected from sensor required to convert in digital form. The conversion of data from analog to digital form is done using an ADC. The output signal generated from the voltage sensor is analog form. Hence, ADC is being used to convert analog value of voltage sensor into digital value. Potentiometer (Variable resistor) is a dynamic resistor whose value can be varied with the help of adjustable knob. With the help of the variable resistor, we can increase or decrease the sensitivity of the voltage sensor which is being interface with ADC.

Principle Of Use

For voltage measurements, a current proportional to the measured voltage must be passed through an external resistor R , which is selected by the user and installed in series with the primary circuit of the transducer.

Advantages

- Excellent accuracy
- Very good linearity
- Low thermal drift
- High immunity to external interference

- Low disturbance in common mode.

Applications

- AC variable speed drives and servo motor drives
- Static converters for DC motor drives
- Battery supplied applications
- Uninterruptible Power Supplies (UPS)
- Power supplies for welding applications.

3.4.10 CURRENT SENSOR

A current sensor is a device that detects and converts current to an easily measured output voltage, which is proportional to the current through the measured path. It can be then utilized to display the measured current in an ammeter or can be stored for further analysis in a data acquisition system or can be utilized for control purpose.

Current measurement is of vital importance in many power and instrumentation systems. Traditionally, current sensing was primarily for circuit protection and control. However, with the advancement in technology, current sensing has emerged as a method to monitor and enhance performance.

Sensing Principles:

When a current flows through a wire or in a circuit, voltage drop occurs. Also, a magnetic field is generated surrounding the current carrying conductor. Both of these phenomena are made use of in the design of current sensors. Thus, there are two types of current sensing: direct and indirect.

Direct Sensing: Direct Sensing involves measuring the voltage drop associated with the current passing through passive electrical components.

Indirect Sensing: Indirect Sensing involves measurement of the magnetic field surrounding a conductor through which current passes.

Application

Knowing the amount of current being delivered to the load can be useful for wide variety of applications. Current sensing is used in wide range of electronic systems, viz., Battery life indicators and chargers, 4-20 mA systems, over-current protection and supervising circuits, current and voltage regulators, DC/DC converters, ground fault detectors, programmable current sources, linear and switch-mode power supplies, communications devices, automotive power electronics, motor speed controls and overload protection, etc.

Advantages

- Cost efficient
- High measurement accuracy
- Measurable current range from very low to medium
- Capability to measure DC or AC current
- Detects the high load current caused by accidental shorts

3.4.11 IR SENSOR

IR sensor is very useful if you are trying to make a obstacle avoider robot or a line follower. In this project we are going to make a simple IR sensor which can detect a object around 6-7 cm. IR sensor is nothing but a diode, which is sensitive for infrared radiation. This infrared transmitter and receiver is called as IR TX-RX pair.



Fig 16: IR Sensor

FEATURES

- Fast response time
- Because it have good range which is fulfill our requirements.
- It is very low cost and can be constructed on general purpose.
- It is of very small size.
- You can increase numbers of transmitter as you want for good result
- Good immunity to ambient light and waves are invisible to eyes.

WORKING OF IR

Working of IR sensor is very simple and working principle is totally based on change in resistance of IR receiver. Here in this sensor we connect IR receiver in reverse bias so it give very high resistance if it is not exposed to IR light. the resistance in this case is in range of Mega ohms, but when IR light reflected back and fall on IR receiver. The resistance of Rx it comes in range between Kilo ohms to hundred of ohms. We convert this change in resistance to change in voltage . Then this voltage is applied to a comparator IC which compare it with a threshold level. if voltage of sensor is more than threshold then output is high else it is low which can be used directly for microcontroller.

Applications

Infrared radiation is the region of the electromagnetic spectrum between microwaves and visible light. In infrared communication an LED transmits the infrared signal as bursts of non-visible light. At the receiving end a photodiode or photoreceptor detects and captures the light pulses, which are then processed to retrieve the information they contain. Some common applications of infrared technology are listed below.

1. Augmentative communication devices
2. Car locking systems
3. Computers
 - a. Mouse
 - b. Keyboards

- c. Floppy disk drives
- d. Printers
- 4. Emergency response systems
- 5. Environmental control systems
 - a. Windows
 - b. Doors
 - c. Lights
 - d. Curtains
 - e. Beds
 - f. Radios
- 6. Headphones
- 7. Home security systems
- 8. Navigation systems
- 9. Signage
- 10. Telephones
- 11. TVs, VCRs, CD players, stereos.

3.4.12 ZIGBEE

ZigBee is a specification for a suite of high level communication protocols used to create personal area networks built from small, low-power digital radios. ZigBee is based on an IEEE 802.15 standard. Though low-powered, ZigBee devices often transmit data over longer distances by passing data through intermediate devices to reach more distant ones, creating a mesh network; i.e., a network with no centralized control or high-power transmitter/receiver able to reach all of the networked devices. The decentralized nature of such wireless ad hoc networks make them suitable for applications where a central node can't be relied upon.

ZigBee protocol features:

- Low duty cycle - Provides long battery life
- Low latency
- Support for multiple network topologies: Static, dynamic, star and mesh

- Direct Sequence Spread Spectrum (DSSS)
- Up to 65,000 nodes on a network
- 128-bit AES encryption – Provides secure connections between devices
- Collision avoidance
- Link quality indication
- Clear channel assessment
- Retries and acknowledgements
- Support for guaranteed time slots and packet freshness

SPECIFICATIONS

The core ZigBee specification defines ZigBee's smart, cost-effective and energy-efficient mesh network. It's an innovative, self-configuring, self-healing system of redundant, low-cost, very low-power and even battery-free nodes that enable ZigBee's unique flexibility, mobility and ease of use.

Description

ZigBee is used in applications that require a low data rate, long battery life, and secure networking. ZigBee has a defined rate of 250 kbit/s, best suited for periodic or intermittent data or a single signal transmission from a sensor or input device. Applications include wireless light switches, electrical meters with in-home-displays, traffic management systems, and other consumer and industrial equipment that requires short-range wireless transfer of data at relatively low rates. The technology defined by the ZigBee specification is intended to be simpler and less expensive than other WPANs, such as Bluetooth or Wi-Fi.

Since ZigBee can be used almost anywhere, is easy to implement and needs little power to operate, the opportunity for growth into new markets, as well as innovation in existing markets, is limitless. Here are some facts about ZigBee:

- With hundreds of members around the globe, ZigBee uses the 2.4 GHz radio frequency to deliver a variety of reliable and easy-to-use standards anywhere in the world.

- Consumer, business, government and industrial users rely on a variety of smart and easy-to-use ZigBee standards to gain greater control of everyday activities.
- With reliable wireless performance and battery operation, ZigBee gives you the freedom and flexibility to do more.
- ZigBee offers a variety of innovative standards smartly designed to help you be green and save money.

Table 5: ZIGBEE COMPARISON

	ZigBee™ 802.15.4	Bluetooth™ 802.15.1	Wi-Fi™ 802.11b	GPRS/GSM 1XRTT/CDMA
Application Focus	Monitoring & Control	Cable Replacement	Web, Video, Email	WAN, Voice/Data
System Resource	4KB-32KB	250KB+	1MB+	16MB+
Battery Life (days)	100-1000+	1-7	.1-5	1-7
Nodes Per Network	255/65K+	7	30	1,000
Bandwidth (kbps)	20-250	720	11,000+	64-128
Range (meters)	1-75+	1-10+	1-100	1,000+
Key Attributes	Reliable, Low Power, Cost Effective	Cost, Convenience	Speed, Flexibility	Reach, Quality

USES

ZigBee protocols are intended for embedded applications requiring low data rates and low power consumption. The resulting network will use very small amounts of power — individual devices must have a battery life of at least two years to pass ZigBee certification.

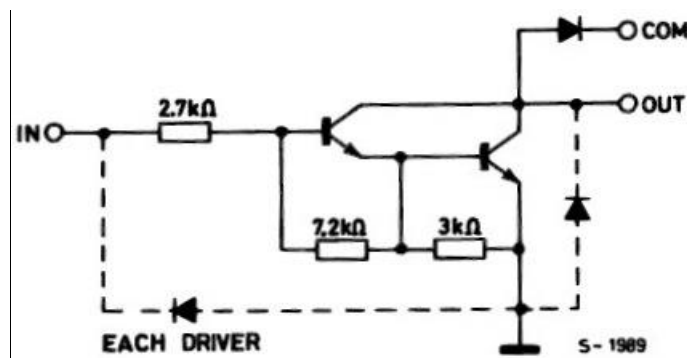
Typical application areas include.

- Home Entertainment and Control — Home automation, smart lighting, advanced temperature control, safety and security, movies and music
- Industrial control
- Embedded sensing
- Medical data collection
- Smoke and intruder warning
- Building automation

3.4.13 ULN2003

The ULN2003 is a monolithic IC consists of seven NPN darlington transistor pairs with high voltage and current capability. It is commonly used for applications such as relay drivers,

motor, display drivers, led lamp drivers, logic buffers, line drivers, hammer drivers and other high voltage current applications. It consists of common cathode clamp diodes for each NPN darlington pair which makes this driver IC useful for switching inductive loads.



The output of the driver is open collector and the collector current rating of each darlington pair is 500mA. Darlington pairs may be paralleled if higher current is required. The driver IC also consists of a 2.7KΩ base resistor for each darlington pair. Thus each darlington pair can be operated directly with TTL or 5V CMOS devices. This driver IC can be used for high voltage applications up to 50V.

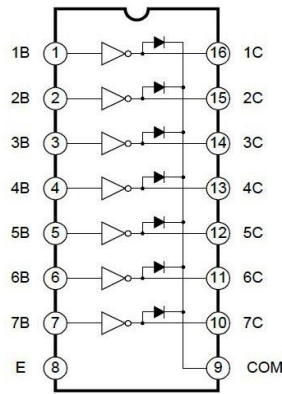


Fig 17: Logic Diagram of ULN2003

Note that the driver provides open collector output, so it can only sink current, cannot source. Thus when a 5V is given to 1B terminal, 1C terminal will be connected to ground via darlington pair and the maximum current that it can handle is 500A. From the above logic diagram we can see that cathode of protection diodes are shorted to 9th pin called COM. So for driving inductive loads, it must connect to the supply voltage.

ULN2003 is widely used in relay driving and stepper motor driving applications.

FEATURES

- * 500mA rated collector current (Single output)
- * High-voltage outputs: 50V
- * Inputs compatible with various types of logic.
- * Relay driver application

3.4.14 RELAY

INTRODUCTION

A relay is an electromechanical switch, which perform ON and OFF operations without any human interaction. General representation of double contact relay is shown in fig. Relays are used where it is necessary to control a circuit by a low-power signal (with complete electrical isolation between control and controlled circuits), or where several circuits must be controlled by one signal.

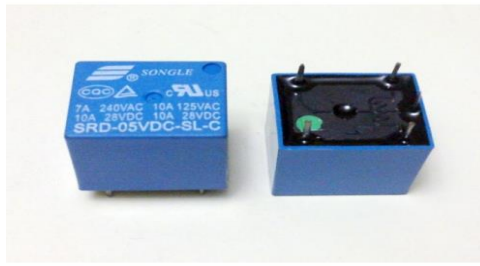


Fig 18: Relay

History

The first relay was invented by Joseph Henry in 1835. The name relay derives from the French noun relays' that indicates the horse exchange place of the postman. Generally a relay is an electrical hardware device having an input and output gate. The output gate consists in one or more electrical contacts that switch when the input gate is electrically excited. It can implement a decoupled, a router or breaker for the electrical power, a negation, and, on the base of the wiring, complicated logical functions containing and, or, and flip-flop. In the past relays had a wide use, for instance the telephone switching or the railway routing and crossing systems. In spite of electronic progresses (as programmable devices), relays are still used in applications where ruggedness, simplicity, long life and high reliability are important factors (for instance in safety applications)

Working

Generally, the relay consists a inductor coil, a spring (not shown in the figure), Swing terminal, and two high power contacts named as normally closed (NC) and normally opened (NO). Relay uses an Electromagnet to move swing terminal between two contacts (NO and NC). When there is no power applied to the inductor coil (Relay is OFF), the spring holds the swing terminal is attached to NC contact.

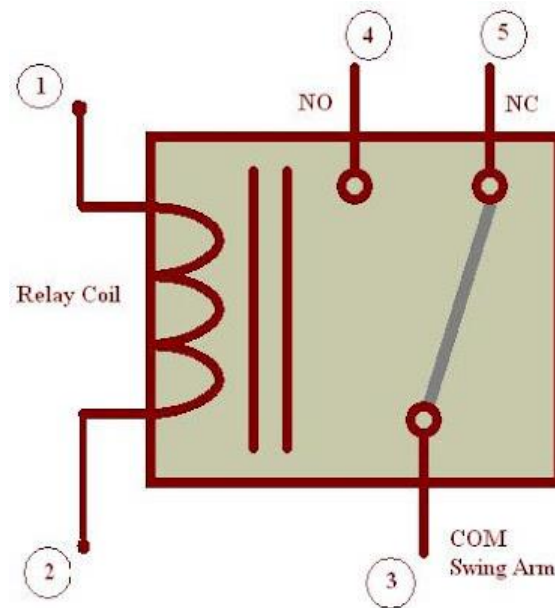


Fig 19: Representation of Relay

Whenever required power is applied to the inductor coil, the current flowing through the coil generates a magnetic field which is helpful to move the swing terminal and attached it to the normally open (NO) contact. Again when power is OFF, the spring restores the swing terminal position to NC.

Advantage of relay:

A relay takes small power to turn ON, but it can control high power devices to switch ON and OFF. Consider an example; a relay is used to control the ceiling FAN at our home. The ceiling FAN may runs at 230V AC and draws a current maximum of 4A. Therefore the power required is $4 \times 230 = 920$ watts. Off course we can control AC, lights, etc., depend up on the relay ratings. Relays can be used to control DC motors in ROBOTICS.

3.4.15 BUZZER

A buzzer or beeper is a signaling device, usually electronic, typically used in automobiles, house hold appliances such as a microwave oven, or game shows. It most commonly consists of a number of switches or sensors connected to a control unit that determines if and which button was pushed or a preset time has lapsed, and usually illuminates a light on the appropriate button or control panel, and sounds a warning in the form of a continuous or intermittent buzzing or

beeping sound. Initially this device was based on an electromechanical system which was identical to an electric bell without the metal gong (which makes the ringing noise). Often these units were anchored to a wall or ceiling and used the ceiling or wall as a sounding board. Another implementation with some AC-connected devices was to implement a circuit to make the AC current into a noise loud enough to drive a loudspeaker and hook this circuit up to a cheap 8-ohm speaker. Nowadays, it is more popular to use a ceramic-based piezoelectric sounder like a Sonalert which makes a high-pitched tone. Usually these were hooked up to “driver” circuits which varied the pitch of the sound or pulsed the sound on and off.

In game shows it is also known as a “lockout system,” because when one person signals (“buzzes in”), all others are locked out from signalling. Several game shows have large buzzer buttons which are identified as “plungers”.



Fig 20: Buzzer

USES

- Annunciator panels
- Electronic metronomes
- Game shows
- Microwave ovens and other household appliances
- Sporting events such as basketball games
- Electrical alarms

3.5 SOFTWARE SPECIFICATIONS

Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuino hardware to upload programs and communicate with them.

WRITING SKETCHES

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension. `.ino`. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom righthand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

NB: Versions of the Arduino Software (IDE) prior to 1.0 saved sketches with the extension `.pde`. It is possible to open these files with version 1.0, you will be prompted to save the sketch with the `.ino` extension on save.

Verify

Checks your code for errors compiling it.



Upload

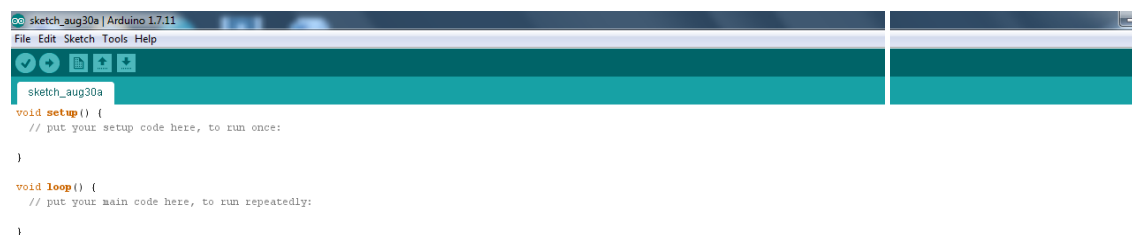
Compiles your code and uploads it to the configured board. See [uploading](#) below for details.

Note: If you are using an external programmer with your board, you can hold down the "shift" key on your computer when using this icon. The text will change to "Upload using Programmer"



New

Creates a new sketch.





Open

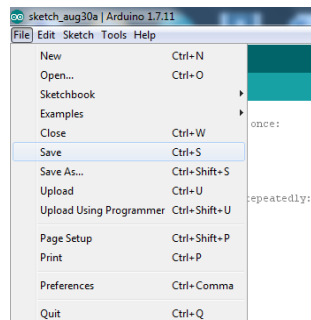
Presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window overwriting its content.

Note: due to a bug in Java, this menu doesn't scroll; if you need to open a sketch late in the list, use the File | Sketchbookmenu instead.



Save

Saves your sketch.



SerialMonitor

Opens the serial monitor.

Additional commands are found within the five menus: File, Edit, Sketch, Tools, Help. The menus are context sensitive, which means only those items relevant to the work currently being carried out are available.

FILE

- *New*

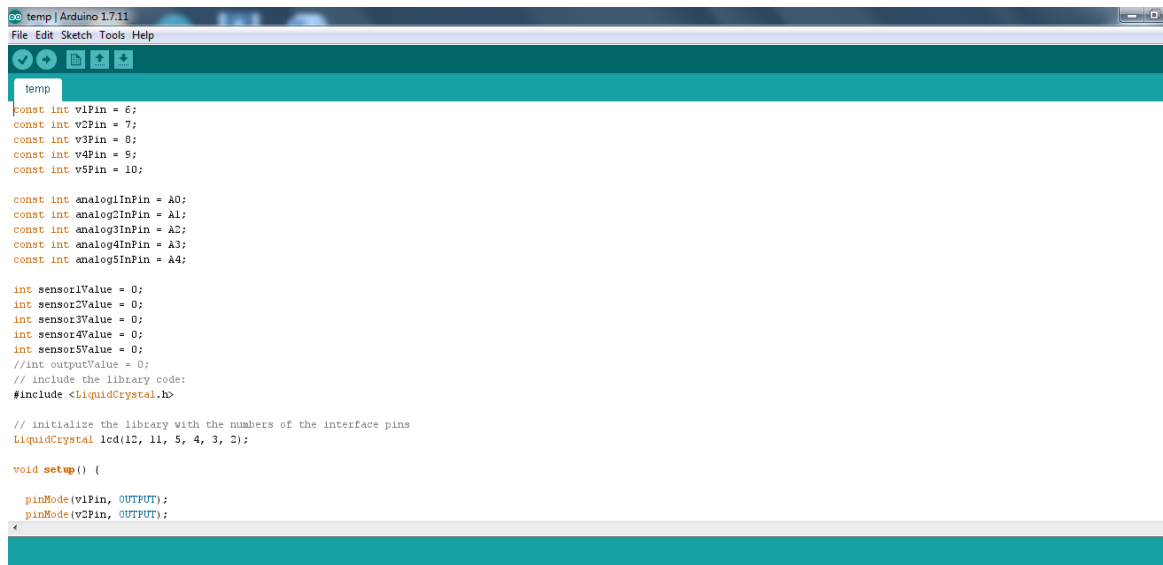
Creates a new instance of the editor, with the bare minimum structure of a sketch already in place.

- *Open*

Allows to load a sketch file browsing through the computer drives and folders.

- *OpenRecent*

Provides a short list of the most recent sketches, ready to be opened.



- *Sketchbook*

Shows the current sketches within the sketchbook folder structure; clicking on any name opens the corresponding sketch in a new editor instance.

- *Examples*

Any example provided by the Arduino Software (IDE) or library shows up in this menu item. All the examples are structured in a tree that allows easy access by topic or library.

- *Close*

Closes the instance of the Arduino Software from which it is clicked.

- *Save*

Saves the sketch with the current name. If the file hasn't been named before, a name will be provided in a "Save as.." window.

- *Saveas...*

Allows to save the current sketch with a different name.

- *PageSetup*

It shows the Page Setup window for printing.

- *Print*

Sends the current sketch to the printer according to the settings defined in Page Setup.

- *Preferences*

Opens the Preferences window where some settings of the IDE may be customized, as the language of the IDE interface.

- *Quit*

Closes all IDE windows. The same sketches open when Quit was chosen will be automatically reopened the next time you start the IDE.

EDIT

- *Undo/Redo*

Goes back of one or more steps you did while editing; when you go back, you may go forward with Redo.

- *Cut*

Removes the selected text from the editor and places it into the clipboard.

- *Copy*

Duplicates the selected text in the editor and places it into the clipboard.

- *Copy for Forum*

Copies the code of your sketch to the clipboard in a form suitable for posting to the forum, complete with syntax coloring.

- *Copy as HTML*

Copies the code of your sketch to the clipboard as HTML, suitable for embedding in web pages.

- *Paste*

Puts the contents of the clipboard at the cursor position, in the editor.

- *Select All*

Selects and highlights the whole content of the editor.

- *Comment/Uncomment*

Puts or removes the // comment marker at the beginning of each selected line.

- *Increase/Decrease Indent*

Adds or subtracts a space at the beginning of each selected line, moving the text one space on the right or eliminating a space at the beginning.

- *Find*

Opens the Find and Replace window where you can specify text to search inside the current sketch according to several options.

- *Find Next*

Highlights the next occurrence - if any - of the string specified as the search item in the Find window, relative to the cursor position.

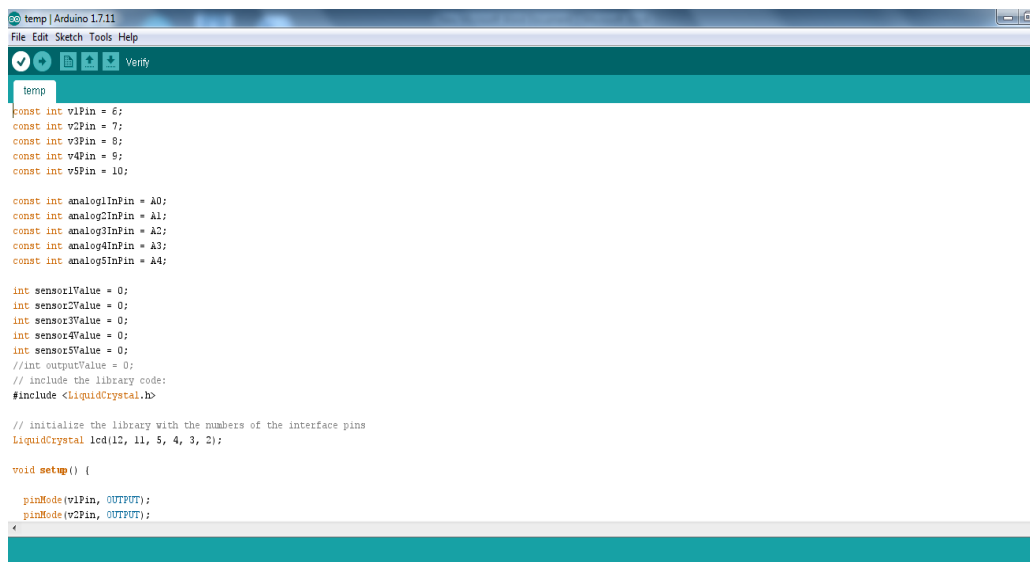
- *Find Previous*

Highlights the previous occurrence - if any - of the string specified as the search item in the Find window relative to the cursor position.

SKETCH

- *Verify/Compile*

Checks your sketch for errors compiling it; it will report memory usage for code and variables in the console area.



```
temp | Arduino 1.7.11
File Edit Sketch Tools Help
[Icons] Verify
temp
const int v1Pin = 6;
const int v2Pin = 7;
const int v3Pin = 8;
const int v4Pin = 9;
const int v5Pin = 10;

const int analog1InPin = A0;
const int analog2InPin = A1;
const int analog3InPin = A2;
const int analog4InPin = A3;
const int analog5InPin = A4;

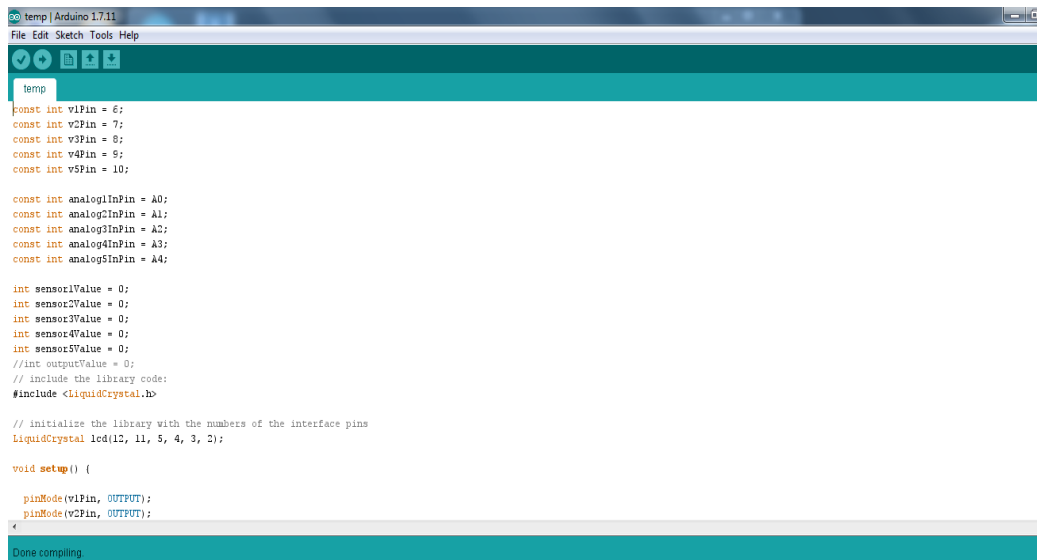
int sensor1Value = 0;
int sensor2Value = 0;
int sensor3Value = 0;
int sensor4Value = 0;
int sensor5Value = 0;
//int outputValue = 0;
// include the library code:
#include <LiquidCrystal.h>

// initialize the library with the numbers of the interface pins
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  pinMode(v1Pin, OUTPUT);
  pinMode(v2Pin, OUTPUT);
```

- *Upload -*

Compiles and loads the binary file onto the configured board through the configured Port.



```
temp
const int v1Pin = 6;
const int v2Pin = 7;
const int v3Pin = 8;
const int v4Pin = 9;
const int v5Pin = 10;

const int analog1InPin = A0;
const int analog2InPin = A1;
const int analog3InPin = A2;
const int analog4InPin = A3;
const int analog5InPin = A4;

int sensor1Value = 0;
int sensor2Value = 0;
int sensor3Value = 0;
int sensor4Value = 0;
int sensor5Value = 0;
//int outputValue = 0;
// include the library code:
#include <LiquidCrystal.h>

// initialize the library with the numbers of the interface pins
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  pinMode(v1Pin, OUTPUT);
  pinMode(v2Pin, OUTPUT);
}
```

- *Upload Using Programmer*

This will overwrite the bootloader on the board; you will need to use Tools > Burn Bootloader to restore it and be able to Upload to USB serial port again. However, it allows you to use the full capacity of the Flash memory for your sketch. Please note that this command will NOT burn the fuses. To do so a Tools -> Burn Bootloader command must be executed.

- *Export Compiled Binary*

Saves a .hex file that may be kept as archive or sent to the board using other tools.

- *Show Sketch Folder*

Opens the current sketch folder.

- *Include Library*

Adds a library to your sketch by inserting #include statements at the start of your code. For more details, see [libraries](#) below. Additionally, from this menu item you can access the Library Manager and import new libraries from .zip files.

- *Add File...*

Adds a source file to the sketch (it will be copied from its current location). The new file appears in a new tab in the sketch window. Files can be removed from the sketch using the tab menu accessible clicking on the small triangle icon below the serial monitor one on the right side of the toolbar.

- **3.5.1 TOOLS**

- *Auto Format*

This formats your code nicely: i.e. indents it so that opening and closing curly braces line up, and that the statements inside curly braces are indented more.

- *Archive Sketch*

Archives a copy of the current sketch in .zip format. The archive is placed in the same directory as the sketch.

- *Fix Encoding & Reload*

Fixes possible discrepancies between the editor char map encoding and other operating systems char maps.

- *Serial Monitor*

Opens the serial monitor window and initiates the exchange of data with any connected board on the currently selected Port. This usually resets the board, if the board supports Reset over serial port opening.

- *Board*

Select the board that you're using. See below for [descriptions of the various boards](#).

- *Port*

This menu contains all the serial devices (real or virtual) on your machine. It should automatically refresh every time you open the top-level tools menu.

- *Programmer*

For selecting a hardware programmer when programming a board or chip and not using the onboard USB-serial connection. Normally you won't need this, but if you're [burning a bootloader](#) to a new microcontroller, you will use this.

- *Burn Bootloader*

The items in this menu allow you to burn a [bootloader](#) onto the microcontroller on an Arduino board. This is not required for normal use of an Arduino or Genuino board but is useful if you purchase a new ATmega microcontroller (which normally come without a bootloader). Ensure that you've selected the correct board from the Boards menu before burning the bootloader on the target board. This command also set the right fuses.

-

Help

Here you find easy access to a number of documents that come with the Arduino Software (IDE). You have access to Getting Started, Reference, this guide to the IDE and other documents locally, without an internet connection. The documents are a local copy of the online ones and may link back to our online website.

- *Find in Reference*

This is the only interactive function of the Help menu: it directly selects the relevant page in the local copy of the Reference for the function or command under the cursor.

3.5.2 SKETCHBOOK

The Arduino Software (IDE) uses the concept of a sketchbook: a standard place to store your programs (or sketches). The sketches in your sketchbook can be opened from the File > Sketchbook menu or from the Open button on the toolbar. The first time you run the Arduino software, it will automatically create a directory for your sketchbook. You can view or change the location of the sketchbook location from with the Preferences dialog.

Beginning with version 1.0, files are saved with a .ino file extension. Previous versions use the .pde extension. You may still open .pde named files in version 1.0 and later, the software will automatically rename the extension to .ino.

Tabs, Multiple Files, and Compilation

Allows you to manage sketches with more than one file (each of which appears in its own tab). These can be normal Arduino code files (no visible extension), C files (.c extension), C++ files (.cpp), or header files (.h).

3.5.3 UPLOADING

Before uploading your sketch, you need to select the correct items from the Tools > Board and Tools > Port menus. The boards are described below. On the Mac, the serial port is probably something like /dev/tty.usbmodem241 (for an Uno or Mega2560 or Leonardo) or /dev/tty.usbserial-1B1 (for a Duemilanove or earlier USB board), or /dev/tty.USA19QW1b1P1.1 (for a serial board connected with a Keyspan USB-to-Serial adapter). On Windows, it's probably COM1 or COM2 (for a serial board) or COM4, COM5, COM7, or

higher (for a USB board) - to find out, you look for USB serial device in the ports section of the Windows Device Manager. On Linux, it should be `/dev/ttyACMx`, `/dev/ttyUSBx` or similar. Once you've selected the correct serial port and board, press the upload button in the toolbar or select the Upload item from the Sketch menu. Current Arduino boards will reset automatically and begin the upload. With older boards (pre-Diecimila) that lack auto-reset, you'll need to press the reset button on the board just before starting the upload. On most boards, you'll see the RX and TX LEDs blink as the sketch is uploaded. The Arduino Software (IDE) will display a message when the upload is complete, or show an error.

When you upload a sketch, you're using the Arduino bootloader, a small program that has been loaded on to the microcontroller on your board. It allows you to upload code without using any additional hardware. The bootloader is active for a few seconds when the board resets; then it starts whichever sketch was most recently uploaded to the microcontroller. The bootloader will blink the on-board (pin 13) LED when it starts (i.e. when the board resets).

3.5.4 LIBRARIES

Libraries provide extra functionality for use in sketches, e.g. working with hardware or manipulating data. To use a library in a sketch, select it from the Sketch > Import Library menu. This will insert one or more `#include` statements at the top of the sketch and compile the library with your sketch. Because libraries are uploaded to the board with your sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete its `#include` statements from the top of your code.

There is a [list of libraries](#) in the reference. Some libraries are included with the Arduino software. Others can be downloaded from a variety of sources or through the Library Manager. Starting with version 1.0.5 of the IDE, you do can import a library from a zip file and use it in an open sketch. See these [instructions for installing a third-party library](#).

To write your own library, see [this tutorial](#).

Third-Party Hardware

Support for third-party hardware can be added to the hardware directory of your

sketchbook directory. Platforms installed there may include board definitions (which appear in the board menu), core libraries, bootloaders, and programmer definitions. To install, create the hardware directory, then unzip the third-party platform into its own sub-directory. (Don't use "arduino" as the sub-directory name or you'll override the built-in Arduino platform.) To uninstall, simply delete its directory.

3.5.5 SERIAL MONITOR

Displays serial data being sent from the Arduino or Genuino board (USB or serial board). To send data to the board, enter text and click on the "send" button or press enter. Choose the baud rate from the drop-down that matches the rate passed to `Serial.begin` in your sketch. Note that on Windows, Mac or Linux, the Arduino or Genuino board will reset (rerun your sketch execution to the beginning) when you connect with the serial monitor.

You can also talk to the board from Processing, Flash, MaxMSP, etc (see the [interfacing page](#) for details).

3.5.6 PREFERENCES

Some preferences can be set in the preferences dialog (found under the Arduino menu on the Mac, or File on Windows and Linux). The rest can be found in the preferences file, whose location is shown in the preference dialog. Since version 1.0.1, the Arduino Software (IDE) has been translated into 30+ different languages. By default, the IDE loads in the language selected by your operating system. (Note: on Windows and possibly Linux, this is determined by the locale setting which controls currency and date formats, not by the language the operating system is displayed in.)

If you would like to change the language manually, start the Arduino Software (IDE) and open the Preferences window. Next to the Editor Language there is a dropdown menu of currently supported languages. Select your preferred language from the menu, and restart the software to use the selected language. If your operating system language is not supported, the Arduino Software (IDE) will default to English. You can return the software to its default setting of selecting its language based on your operating system by selecting System

Default from the Editor Language drop-down. This setting will take effect when you restart the Arduino Software (IDE). Similarly, after changing your operating system's settings, you must restart the Arduino Software (IDE) to update it to the new default language.

3.5.7 BOARDS

The board selection has two effects: it sets the parameters (e.g. CPU speed and baud rate) used when compiling and uploading sketches; and sets the file and fuse settings used by the burn bootloader command. Some of the board definitions differ only in the latter, so even if you've been uploading successfully with a particular selection, you'll want to check it before burning the bootloader. You can find a comparison table between the various boards [here](#).

Arduino Software (IDE) includes the built in support for the boards in the following list, all based on the AVR Core. The [Boards Manager](#) included in the standard installation allows to add support for the growing number of new boards based on different cores like Arduino Due, Arduino Zero, Edison, Galileo and so on.

3.6 IMPLEMENTATIONS

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SoftwareSerial.h>

#define SCREEN_WIDTH 128 // OLED display width, in pixels
#define SCREEN_HEIGHT 64 // OLED display height, in pixels

// Declaration for an SSD1306 display connected to I2C (SDA, SCL pins)
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -
1);
#define temp 17
const int curr = 26;

#define volt 22
#define fan 18
```

```
#define Load 19
#define Buz 16

int buttonState = 1;
int i;

void setup() {
  Serial.begin(9600);
  Serial2.begin(9600);
  pinMode(Buz, OUTPUT);
  pinMode(Load, OUTPUT);
  pinMode(fan, OUTPUT);
  pinMode(temp, INPUT_PULLUP);
  delay(200);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Address 0x3D
    for 128x64
      Serial.println(F("SSD1306 allocation failed"));
      for (;;)
        ;
  }
  delay(1000);
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0, 0);

  display.println("DESIGN OF SENSOR NETWORK WITH LONG
DISTANCE COMMUNICATION FOR APPLICATION IN BUILDING
ENERGY MANAGEMENT SYSTEMS");
  display.display();
  delay(2000);

  digitalWrite(Buz, LOW);
  digitalWrite(Load, HIGH);
  digitalWrite(fan, LOW);

  display.clearDisplay();
  display.display();
}

void loop() {
  display.clearDisplay();

  delay(500)
```

```
buttonState = digitalRead(temp);
float Current = analogRead(curr);
display.setCursor(0, 0);
display.print("Current :");
display.print(Current);
display.display();
Serial2.println("Current :");
Serial2.println(Current);
delay(1000);

float Voltage = digitalRead(volt);
display.setCursor(0, 10);
display.print("Voltage :");
display.print(Voltage);
display.display();
Serial2.println("Voltage :");
Serial2.println(Voltage);
delay(1000);
if (buttonState == LOW) {
    i++;
    digitalWrite(fan, HIGH);
    digitalWrite(Load, HIGH);
    display.setCursor(0, 20);
    display.print("METER READING :");
    display.print(i);
    display.display();
    delay(1000);

    if (i > 10 && i < 15) {
        display.setCursor(0, 40);
        display.println("Please Pay your bill");
        display.display();
        Serial2.println("Please Pay your bill");
        delay(300);
        digitalWrite(fan, LOW);
        digitalWrite(Buz, HIGH);
        delay(300);
        digitalWrite(Buz, LOW);
        delay(300);
        digitalWrite(Buz, HIGH);
        delay(300);
        digitalWrite(Buz, LOW);

        delay(300);
```

```
digitalWrite(Buz, HIGH);
delay(300);
digitalWrite(Buz, LOW);
delay(300);
digitalWrite(Buz, HIGH);
delay(300);
digitalWrite(Buz, LOW);
delay(300);
digitalWrite(Buz, HIGH);
delay(300);
digitalWrite(Buz, LOW);
delay(300);
digitalWrite(Buz, HIGH);
delay(300);
digitalWrite(Buz, LOW);
delay(300);
digitalWrite(Buz, HIGH);
delay(300);
digitalWrite(Buz, LOW);
delay(300);
digitalWrite(Buz, HIGH);
delay(300);
digitalWrite(Buz, LOW);
delay(300);
Serial2.println("Please Pay your bill");
delay(1000);

} else if (i > 15) {
  display.setCursor(0, 40);
  display.println("Power Disconnected....");
  display.display();
  digitalWrite(Buz, HIGH);
  digitalWrite(fan, LOW);
  digitalWrite(Load, LOW);
  delay(1000);
  digitalWrite(Buz, LOW);
  Serial2.println("Power Disconnected....");
  delay(1000);
}
}
}
```

3.7 PHOTOGRAPHS

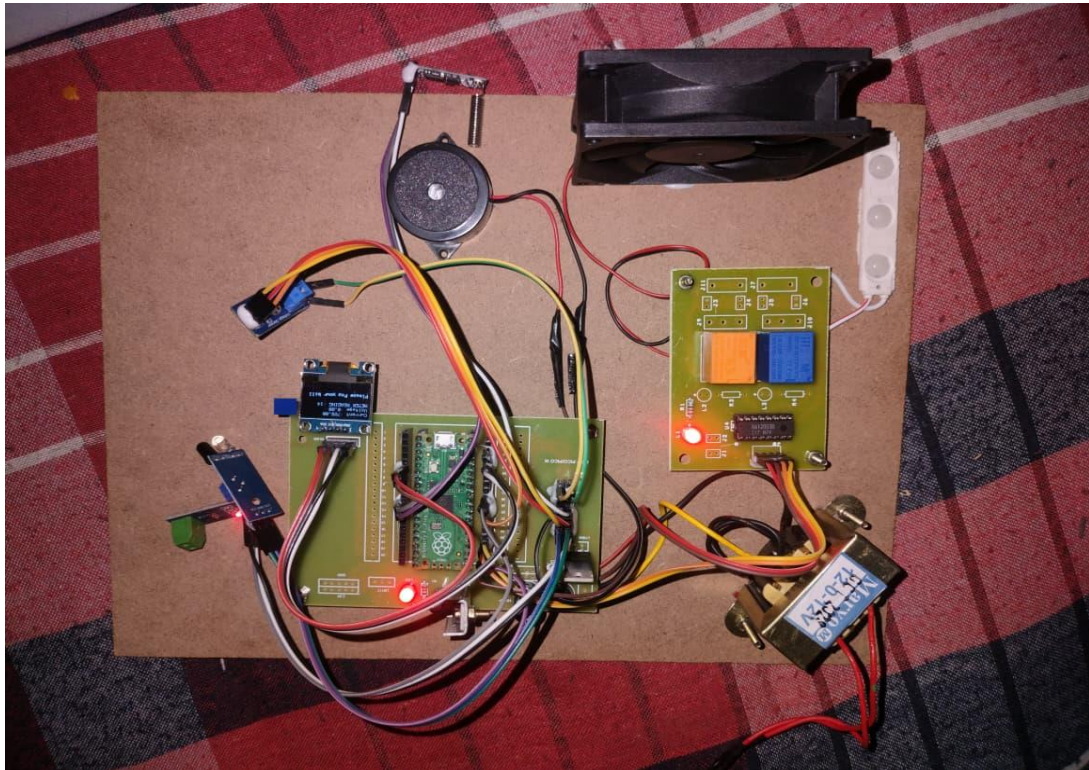


Fig 21: Photograph of Result

CHAPTER 4

RESULT AND DESCRIPTION

The implemented sensor network was tested under various environmental conditions to evaluate its communication range, reliability, and energy efficiency. The results confirm that the system successfully achieves long-distance communication of up to 1000 meters using Zigbee technology, maintaining stable connectivity with low latency. Packet delivery ratio remained high throughout the testing phase, indicating reliable data transmission even in the presence of minor physical obstructions typically found in building environments. The optimization of communication parameters, including transmission power, bandwidth, and data rate, played a crucial role in enhancing system performance. By selecting appropriate configurations, the network achieved a balance between communication range and power consumption. The results show a noticeable reduction in energy usage compared to conventional sensor networks, thereby extending the operational lifetime of sensor nodes. The environmental sensor nodes accurately captured and transmitted real-time data such as temperature, humidity, and light intensity. The integration with the Ubidots IoT platform provided efficient data visualization through dashboards, enabling users to monitor conditions remotely and make informed decisions. The system also demonstrated consistent performance with minimal data loss and acceptable transmission delays. Scalability testing indicated that additional sensor nodes could be incorporated into the network without significant degradation in performance, highlighting the flexibility of the proposed architecture.

CHAPTER 5

CONCLUSION

In this paper, a sensor network was developed to address energy efficiency and pollution issues. It is developed and deployed in our school for building monitoring. The developed network is based on a hierarchical structure composed of generic and modular sensor nodes, gateway nodes, and base station. During the design of this network, we highlighted challenges related to energy constraints, large-scale network, development complexity, and costs. To overcome these challenges, we proposed a MDD approach. We managed to implement a draft of this approach by using GME tool. Our approach allows to estimate sensor node lifetime, to monitor sensor network in real time, and to generate simulations. The major contributions of this work are related to the development of a generic sensor network for smart buildings in order to increase their energy efficiency. As second contribution, we have identified challenges related to sensor network design and we provide some answers to these challenges.

REFERENCE

1. Akyildiz, I.F., Su, W., Sankarasubramaniam, Y. and Cayirci, E. (2002) 'Wireless sensor networks: a survey', *Computer Networks*, Vol. 38, No. 4, pp.393–422.
2. Al Saad, M., Fehr, E., Kamenzky, N. and Schiller, J. (2008) 'ScatterClipse: a model-driven tool-chain for developing, testing, and prototyping wireless sensor networks', in *Parallel and Distributed Processing with Applications*, 2008, ISPA'08, International Symposium on, pp.871–885, December.
3. Atkinson, C. and Kuhne, T. (2003) 'Model-driven development: a metamodeling foundation', *IEEE Software*, Vol. 20, No. 5, pp.36–41.
4. Boonma, P. and Suzuki, J. (2011) 'Model-driven performance engineering for wireless sensor networks with feature modeling and event calculus', in *Proceedings of the 3rd Workshop on Biologically Inspired Algorithms for Distributed Systems*, June, pp.17–24.
5. Borges, L.M., Velez, F.J. and Lebres, A.S. (2014) 'Survey on the characterization and classification of wireless sensor network applications', *IEEE Communications Surveys & Tutorials*, Vol. 16, No. 4, pp.1860–1890.
6. Digi (2017) Digi XBee s2c 802.15.4 rf modules, Technical report, Digi International Singapore [online] https://www.digi.com/pdf/ds_xbee-s2c-802-15-4.pdf, (accessed 30 January 2018).
7. Gezer, C., Buratti, C., & Verdone, R. (2011). An IEEE 802.15.4 Wireless Sensor Network for Industrial Applications. IEEE. Testoon (2015).
8. Karl, H., & Willig, A. (2005). *Protocols and Architectures for Wireless Sensor Networks*. Wiley.